CSE 270 Sports Analytics

Statistical Analysis of Networks

Habet Madoyan

Types of analysis

- Descriptive Statistics for networks, such as network density
- Descriptive statistics for network elements, nodes
- Cluster analysis, community detection, grouping nodes together
- Mathematical and statistical models for network behavior

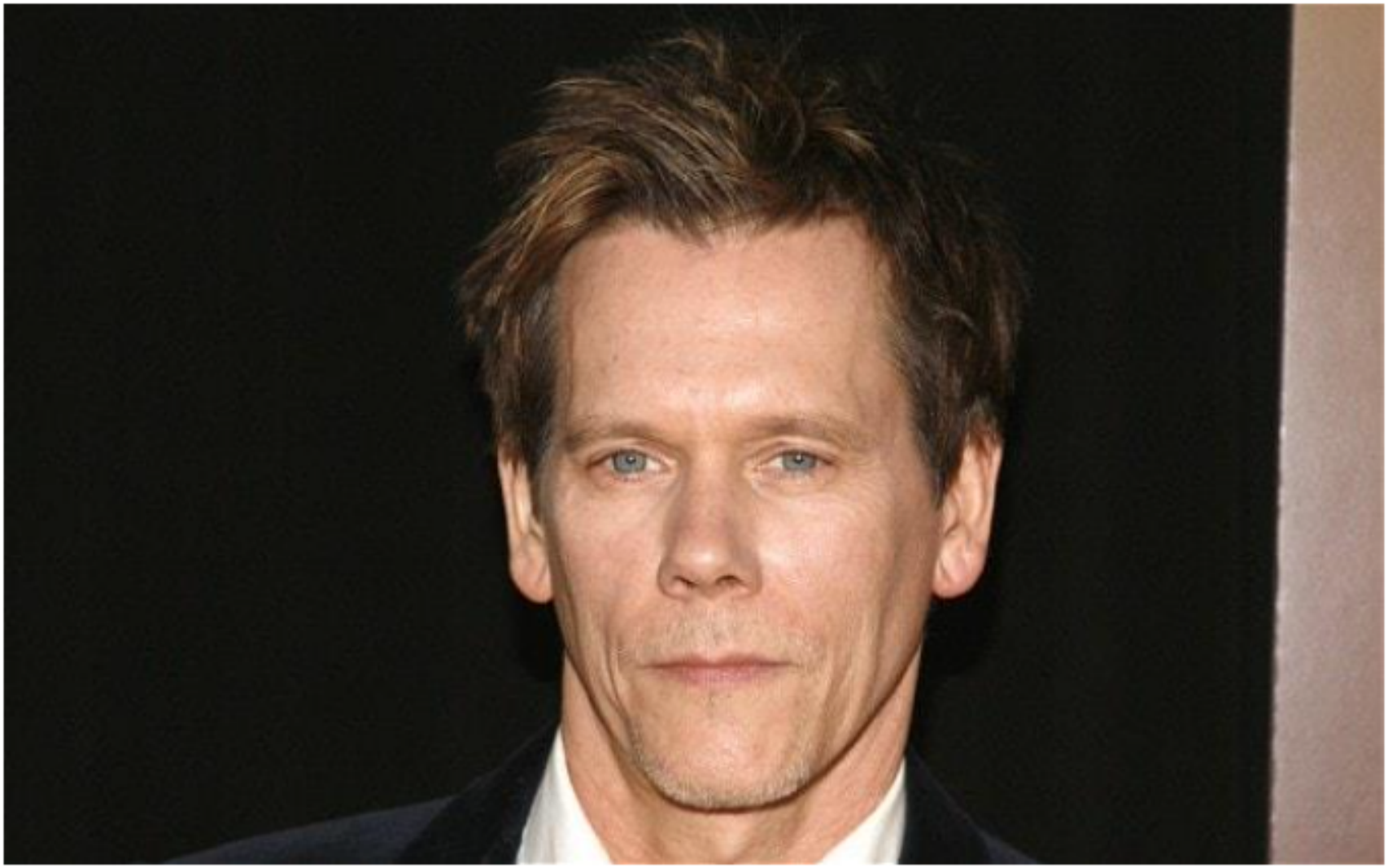
Definition

In network analysis we are going to use **geodesic distance** to measure the distance between two nodes.

In graph theory the **distance** between two vertices in a graph is the number of edges in a shortest path (also called a **graph geodesic**) connecting them. This is also known as the **geodesic distance.**

Who is this guy ?

Kevon Bacon, Played in 86 movies



http://www.imdb.com/name/nm0000102/

Bacon Number

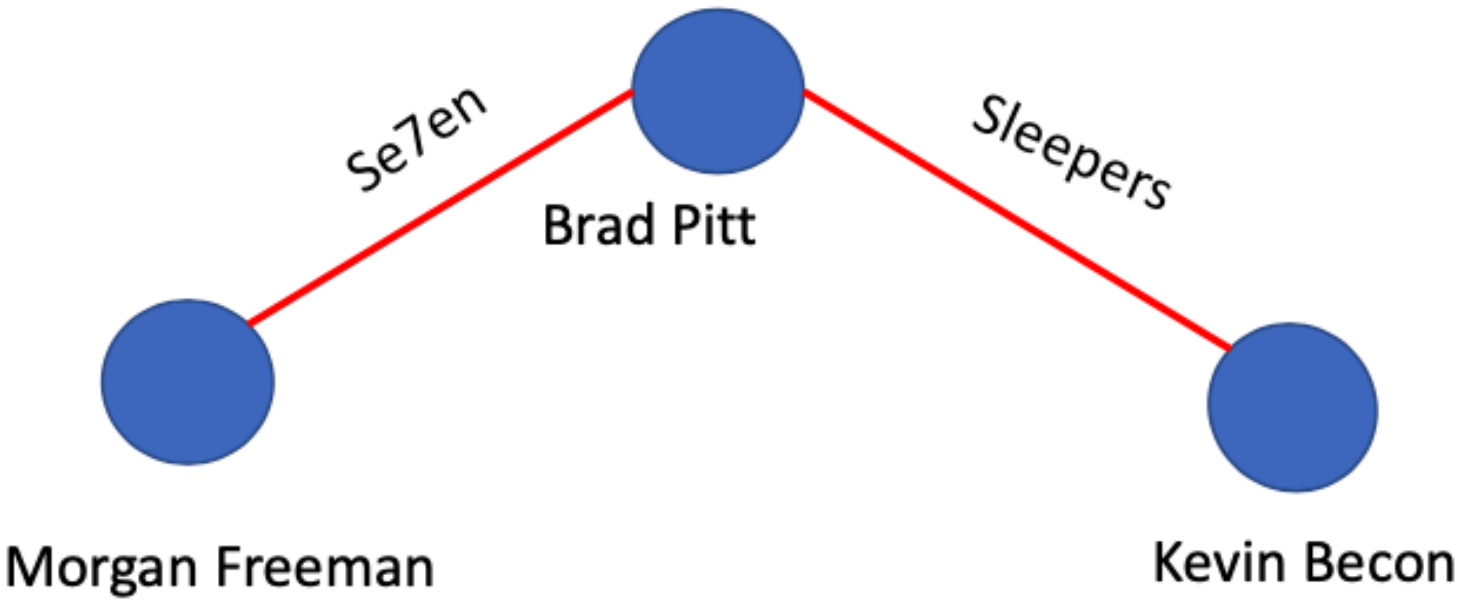
Bacon number is a geodesic distance between an Hollywood actor and Kevin Bacon.

No one in Hollywood has Bacon number greater than 6

https://oracleofbacon.org/ – Social network of movie actors

Bacon Number

Bacon Number of Morgan Freeman

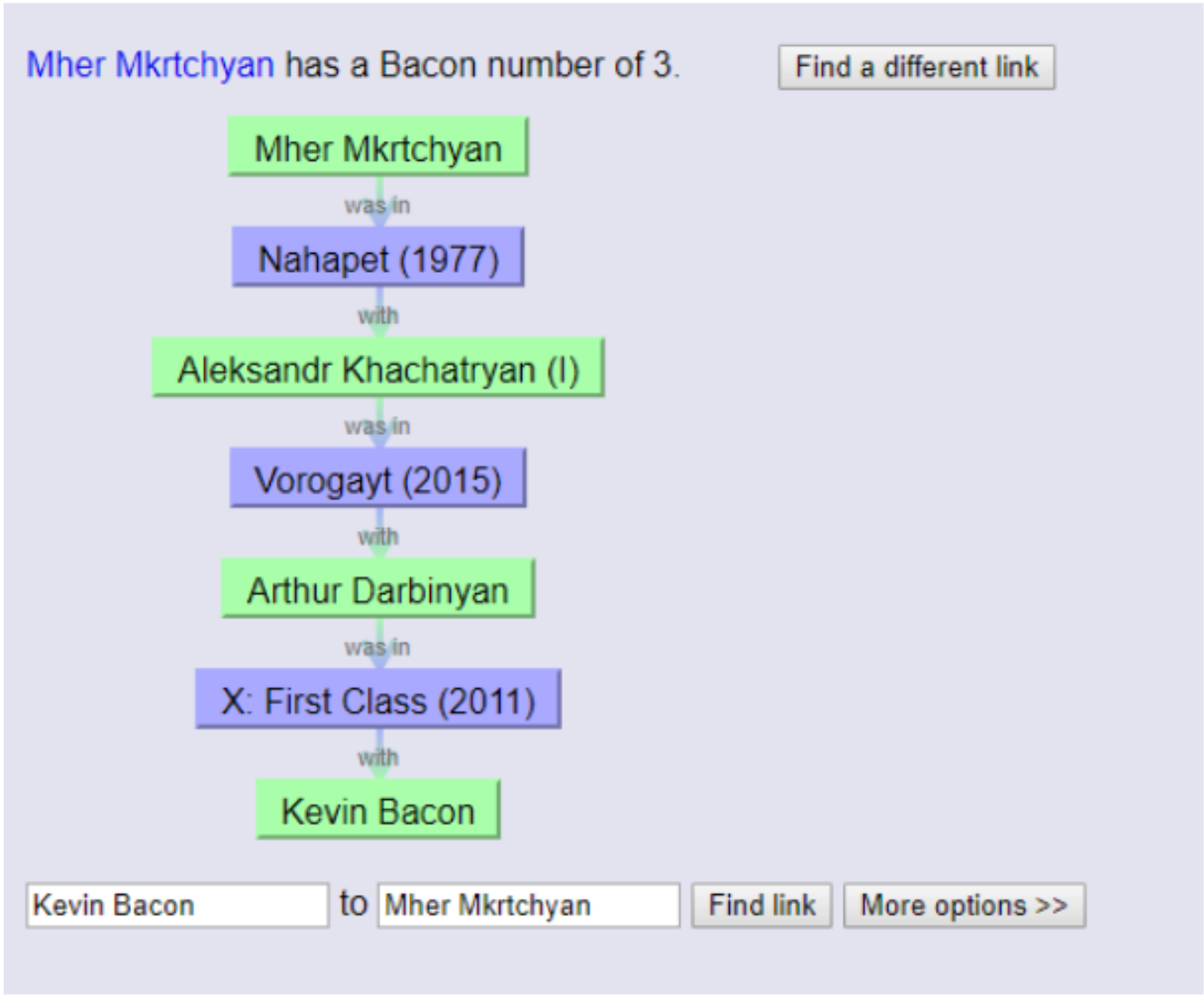


The geodesic distance between Morgan Freeman and Kevin Bacon is 2

Bacon Number

What will be Frunzik's Bacon number ?

Mher Mkrtchyan



Network density

- Density shows how interconnected are the nodes within the network.
- Density is the proportion of observed ties (also called edges, arcs, or relations) in a network to the maximum number of possible ties.
- Thus, density is a ratio that can range from 0 to 1.
- The closer to 1 the density is, the more interconnected is the network.

Network Density

For non directed network, density is calculated with the following formula

$$\frac{2*L}{k(k-1)}$$

For undirected network the total number of ties is equal to

$$\frac{k(k-1)}{2}$$

Where

L is the total number of ties (edges),

k is the total number of nodes/vertices

Network Density

For directed network, density is calculated with the following formula

$$\frac{L}{k(k-1)}$$

Where

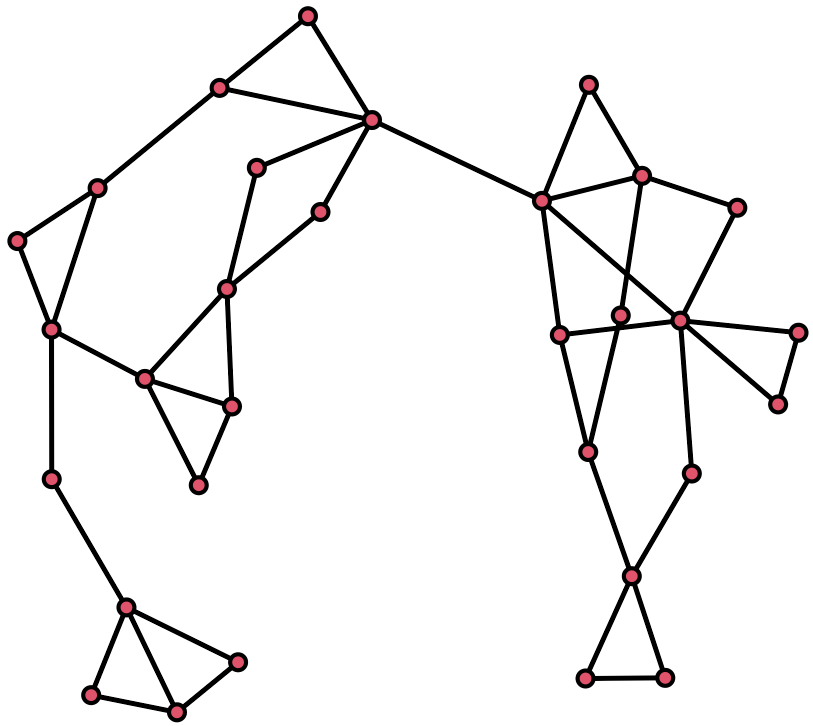
L is the total number of ties (edges),

k is the total number of nodes/vertices

Network size: Un-directed graph

```
library(UserNetR)
library(network)
library(igraph)
library(intergraph)
library(ggplot2)
```

```
set.seed(1)
data(Moreno)
plot(Moreno)
```



Network density: undirected graph

Size of Network: How many actors/nodes are there

```
network.size(Moreno)
```

```
## [1] 33
```

Calculate the density

```
# total number of possible edges
33*(33-1)/2
```

```
## [1] 528
```

```
# Density
2*46/(33*(33-1))
```

```
## [1] 0.08712121
```

```
sna::gden(Moreno)
```

```
## [1] 0.08712121
```

Network's diameter

Network diameter shows how compact is it, given its size and degree of interconnectness.

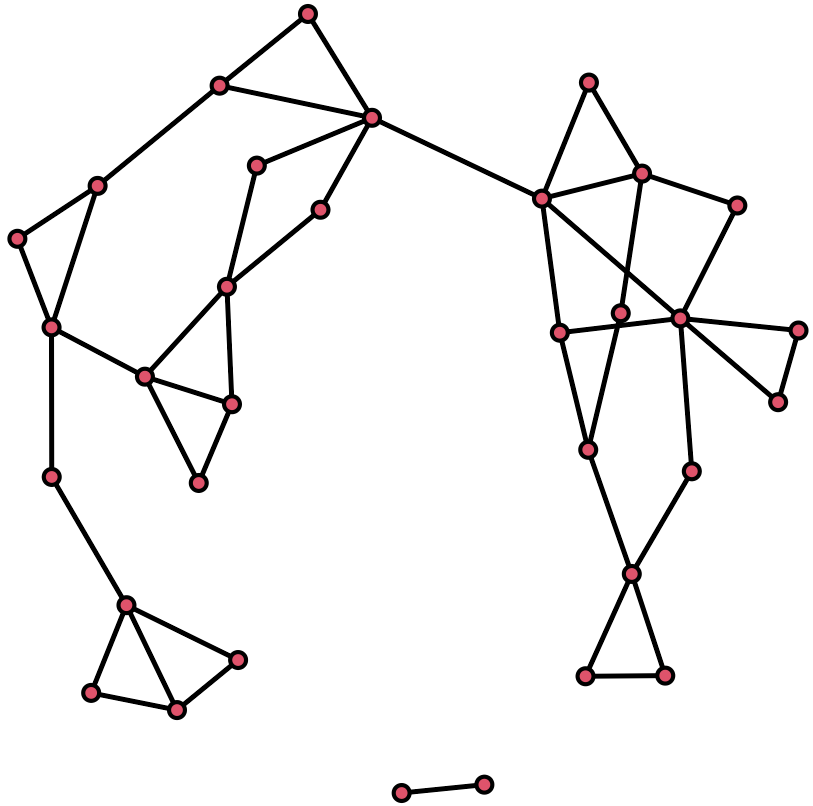
The diameter then for an entire network is the longest of the shortest paths across all pairs of nodes.

The diameter cannot be calculated if there are separate subgroups or components of the graph

Subgroup components

Not for all the nodes we can calculate the shortest path.

```
set.seed(1)
plot(Moreno)
```



Network diameter

Calculate the diameter for the largest component

```
lgc <- sna::component.largest(Moreno,result="graph")
gd <- sna::geodist(lgc)
max(gd$gdist)
```

```
## [1] 11
```

Average path-length

- In a network, the mean path length is the average shortest path between two nodes. Let d_{ij} be the length of the shortest path between nodes i and j.
- In a network, the length of a path is the number of edges that the path contains. The shortest path between two points is called geodesic.
- The mean path length is the average of the shortest path length, averaged over all pairs of nodes. For an undirected graph of N nodes, the mean path length is

$$l = \frac{2}{N(N-1)} \sum_{\mathrm{i} \neq \mathrm{j}} d_{ij}$$

For directed graph

$$l = \frac{1}{N(N-1)} \sum_{\mathrm{i} \neq \mathrm{j}} d_{ij}$$

If two nodes are disconnected, meaning there is no path between them, then the path length between them is infinite. As a consequence, if a network contains disconnected components (collections of nodes that have no paths between them), then the mean path length l also diverges to infinity. One way to avoid this problem is to calculate l only from nodes in the largest connected component, or just ignore unconnected nodes.

Relationships in networks

Create both network and igraph objects

```
mat <- rbind(c(0,1,1,0,0),c(0,0,1,1,0),c(0,1,0,0,0),
             c(0,0,0,0,0),c(0,0,1,0,0))
rownames(mat) <- c("A","B","C","D","E")
colnames(mat) <- c("A","B","C","D","E")
net <- network(mat, matrix.type="adjacency")
net_i <- graph_from_adjacency_matrix(mat)
```

Relationships in networks

Calculating average path length

```
d <- shortest.paths(net_i, v=V(net_i), to=V(net_i))
d
```

```
##   A B C D E
## A 0 1 1 2 2
## B 1 0 1 1 2
## C 1 1 0 2 1
## D 2 1 2 0 3
## E 2 2 1 3 0
```

```
sum(d[upper.tri(d)])
```

```
## [1] 16
```

```
16*2/(5*(5-1))
```

```
## [1] 1.6
```

For Moreno dataset

```
Moreno_i <- asIgraph(Moreno)
edge_density(Moreno_i)
```

```
## [1] 0.08712121
```

```
mean_distance(Moreno_i, directed=F)
```

```
## [1] 4.409871
```

Is the graph random ?

Is the graph random ?

Is the graph random ?

Random graphs

Generating random graphs is an important method for investigating how likely or unlikely other network metrics are likely to occur given certain properties of the original graph. The simplest random graph is one that has the same number of vertices as your original graph and approximately the same density as the original graph.

We will create an ***Erdos-Renyi Random Graph Model***

Is the graph random ?

Creating a random graph with the same number of nodes and edge density as our Moreno network.

We want to find if the average path length of the Moreno graph is random

```
# Number of Nodes
gorder(Moreno_i)
```

```
## [1] 33
```

```
# Edge density
edge_density(Moreno_i)
```

```
## [1] 0.08712121
```

Is the graph random ?

- Generate a random graph using the function erdos.renyi.game().
- The first argument n should be the number of nodes of the graph g which can be calculated using gorder(),
- The second argument p.or.m can be either the density or the number of edges.
- The final argument is set as type=‘gnp’ to tell the function that you are using the density of the graph to generate a random graph.

```
i_random <- erdos.renyi.game(n = gorder(Moreno_i),
                p.or.m = edge_density(Moreno_i),
                type = "gnp")
```

Is the graph random ?

```
{r. fig.height = 6} plot(i_random)
```

Is the graph random ?

Now lets generate 1,000 random graphs and calculate average path length for each of them

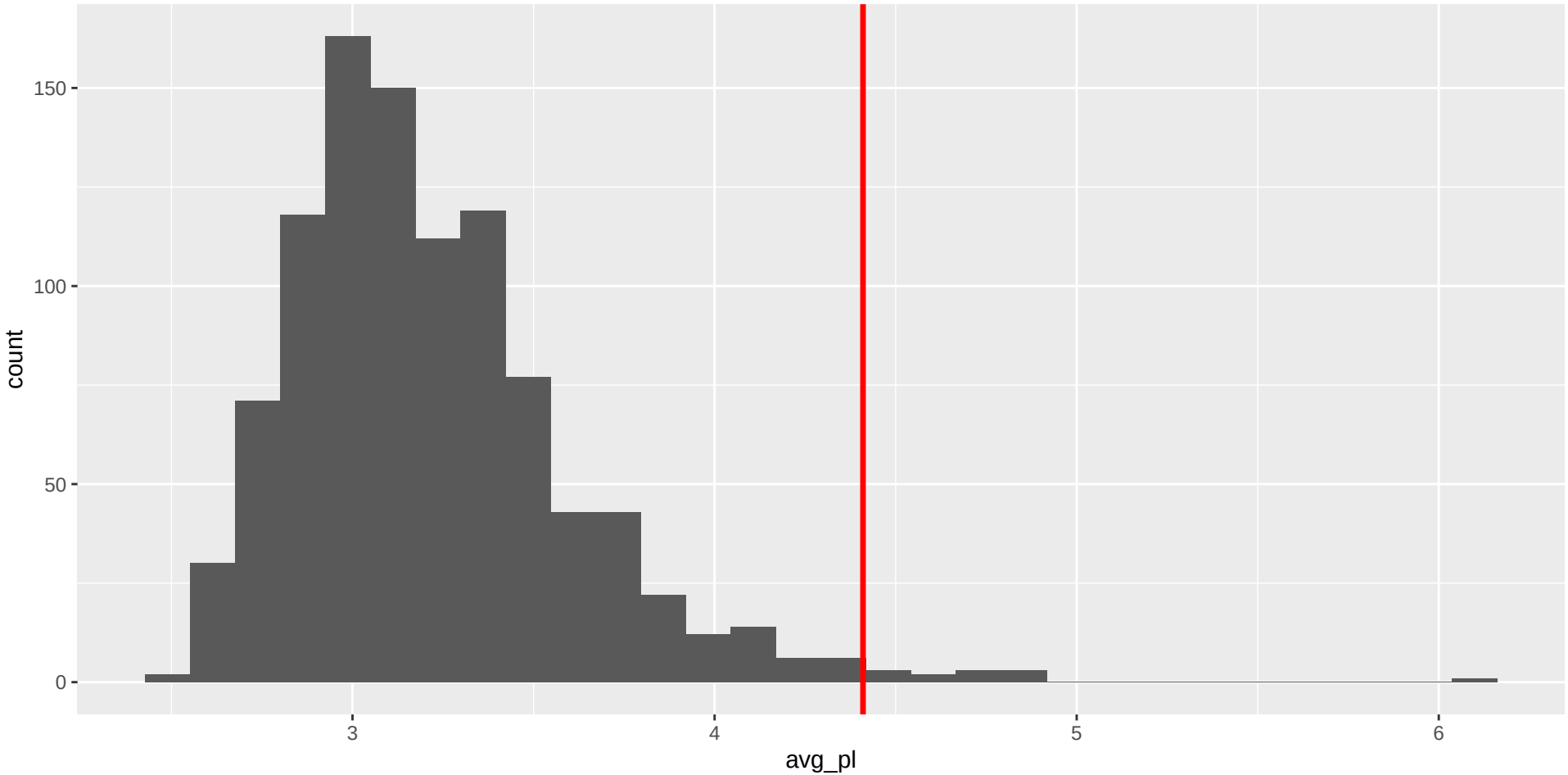
```
i_list <- list()

for(i in 1:1000){
  i_list[[i]] <- erdos.renyi.game(n = gorder(Moreno_i),
                      p.or.m = edge_density(Moreno_i),
                      type = "gnp")
}
```

```
avg_pl <- unlist(lapply(i_list, mean_distance, directed = FALSE))
```

Is the graph random ?

```
ggplot()+geom_histogram(aes(x=avg_pl))+
  geom_vline(xintercept = mean_distance(Moreno_i, directed=F),
             size=1.2, color="red")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Is the graph random ?

Probabiltiy that in a random graph with the given parameters you would have average path length less than for Moreno graph is:

```
mean(avg_pl < mean_distance(Moreno_i, directed=F))
```

```
## [1] 0.988
```

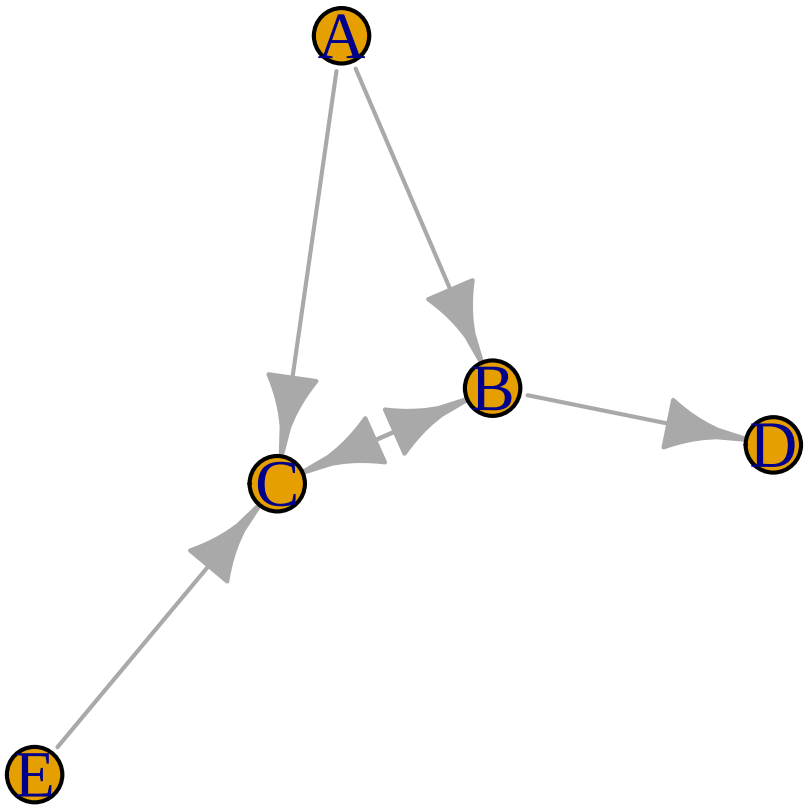
It is random !!!

Relationships between vertices

Relationships between vertices

Relationships in networks

```
plot(net_i)
```



Relationships in networks

- Subset each vertex from which each edge originates by using head_of().
- This function takes two arguments, the first being the graph object and the second the edges to examine. For all edges you can use E(g).

```
E(net_i)
```

```
## + 6/6 edges from 59f5057 (vertex names):
## [1] A->B A->C B->C B->D C->B E->C
```

Relationships in networks

```
head_of(net_i, E(net_i))
```

```
## + 6/5 vertices, named, from 59f5057:
## [1] B C C D B C
```

```
table(head_of(net_i, E(net_i)))
```

```
##
## 2 3 4
## 2 3 1
```

Relationships in networks

tail_of() is going to show the beginning node of each edge

```
tail_of(net_i, E(net_i))
```

```
## + 6/5 vertices, named, from 59f5057:
## [1] A A B B C E
```

```
table(tail_of(net_i, E(net_i)))
```

```
##
## 1 2 3 5
## 2 2 1 1
```

Relationships in networks

We might be also interested if two vertices are connected with each other

Is there an edge between * A and B * B and C and so on

```
net_i['A','B']
```

```
## [1] 1
```

```
net_i['B','C']
```

```
## [1] 1
```

```
net_i['E','B']
```

```
## [1] 0
```

Is there an edge between A and B, A and C

```
net_i['A',c('B', 'C')]
```

```
## B C
## 1 1
```

Relationships in networks

We can use function incident() to identify those edges that go in to the given vertex, or go out from the given vertex, or both

```
incident(net_i, "A", mode = c("out"))
```

```
## + 2/6 edges from 59f5057 (vertex names):
## [1] A->B A->C
```

```
incident(net_i, "A", mode = c("in"))
```

```
## + 0/6 edges from 59f5057 (vertex names):
```

Relationships in networks

Often in network analysis it is important to explore the patterning of connections that exist between vertices. One way is to identify neighboring vertices of each vertex. You can then determine which neighboring vertices are shared even by unconnected vertices indicating how two vertices may have an indirect relationship through others.

Relationships in networks

- Identify neighbors for vertex A That are receiving an edge from A
- Identify vertices that are sending edges to C

```
neighbors(net_i, 'A', mode = c('all'))
```

```
## + 2/5 vertices, named, from 59f5057:
## [1] B C
```

```
neighbors(net_i, 'C', mode = c('in'))
```

```
## + 3/5 vertices, named, from 59f5057:
## [1] A B E
```

Relationships in networks

Identify any verteces that receive edge from vertex C and direct an edge to vertex B

```
A <- neighbors(net_i, 'A', mode = c('out'))
D <- neighbors(net_i, 'D', mode = c('in'))
intersection(A,D)
```

```
## + 1/5 vertex, named, from 59f5057:
## [1] B
```

Relationships in networks

- The inter-connectivity of a network can be assessed by examining the number and length of paths between vertices.
- A path is simply the chain of connections between vertices. The number of intervening edges between two vertices represents the geodesic distance between vertices.
- Vertices that are connected to each other have a geodesic distance of 1.
- Those that share a neighbor in common but are not connected to each other have a geodesic distance of 2 and so on. In directed networks, the direction of edges can be taken into account.

Relationships in networks

```
farthest_vertices(net_i)
```

```
## $vertices
## + 2/5 vertices, named, from 59f5057:
## [1] E D
##
## $distance
## [1] 3
```

Relationships in networks

Identify the longest path

```
get_diameter(net_i)
```

```
## + 4/5 vertices, named, from 59f5057:
## [1] E C B D
```

Relationships in networks

Identify vertices that are reachable within two connections from vertex A

```
ego(net_i, 2, 'A', mode = c('out'))
```

```
## [[1]]
## + 4/5 vertices, named, from 59f5057:
## [1] A B C D
```

Identify any vertices that can reach vertex D within two connections

```
ego(net_i, 2, 'D', mode = c('in'))
```

```
## [[1]]
## + 4/5 vertices, named, from 59f5057:
## [1] D B A C
```

Node's prominence/importance

Node's prominence/importance

Node's prominence/importance

- We will examine the location of each node to understand how important it is for the network.
- We analyze the prominence of the nodes/actors. An actor is prominent if the ties of the actor make that actor visible to the other members in the network.
- We will look at the node centrality measures for non-directed networks, and node prominence measure called prestige for directed networks.
- Prestige and centrality will be used to calculate network-level centralization measures.

Degree Centrality

- The simplest measure of centrality by far is based on the notion that a node that has more direct ties is more prominent than nodes with fewer or no ties.
- Degree centrality thus, is simply the degree of each node.
- The degree of a node is the number of ties it has with other nodes.

Degree centrality

Use Package sna

```
net %v% "vertex.names"
```

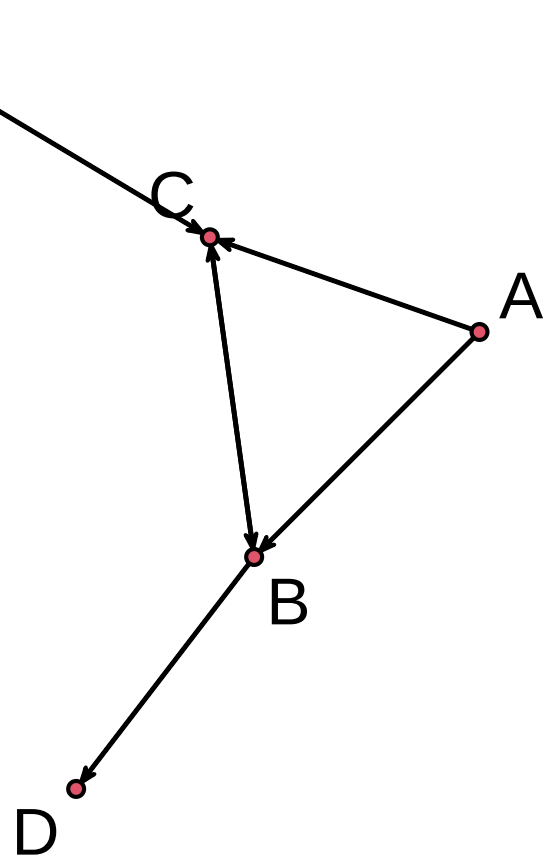
```
## [1] "A" "B" "C" "D" "E"
```

Degree centrality

```
sna::degree(net)
```

```
## [1] 2 4 4 1 1
```

```
plot(net, displaylabels=T)
```



Indegree centrality

Measures number of the edges that come to the given node.

within function degree, specify cmode="indegree"

To get the number of the edges that goes from the node, use cmode="outdegree"

```
sna::degree(net, cmode="indegree")
```

```
## [1] 0 2 3 1 0
```

```
net %v% "vertex.names"
```

```
## [1] "A" "B" "C" "D" "E"
```

```
sna::degree(net, cmode="outdegree")
```

```
## [1] 2 2 1 0 1
```

```
net %v% "vertex.names"
```

```
## [1] "A" "B" "C" "D" "E"
```

Indegree centrality

- The farness/peripherality of a node v is defined as the sum of its distances to all other nodes
- The closeness is defined as the inverse of the farness.
- For comparison purpose, we can standardize the closeness by dividing by the maximum possible value 1/(n - 1)
- If there is no (directed) path between vertex j and i then the total number of vertices is used in the formula instead of the path length.
- The more central a node is, the lower its total distance to all other nodes
- Higher the value, more important is the node

Closeness Centrality

Instead of examining only the direct connections of the nodes, we can focus on how close each node is to everyone else.

Nodes are more prominent/important to the extent they are close to all other nodes in the network

$$C_c(n_i) = [\sum_{j=1}^{g} d(n_i, n_j)]^{-1}$$

Where $d(n_i, n_j)$ is the geodesic distance between nodes.

Closeness centrality, then, is the inverse of the sum of all the distances between node i and all the other nodes in the network.

Closeness Centrality

Commonly normalized version of the Closeness centrality is measured, so the CC is adjusted by the number of the nodes in the graph

$$C_c(n_i) = \frac{N-1}{\sum_{\mathrm{j}=1}^{g} d(n_i, n_j)}$$

Where N is the number of the nodes in the graph

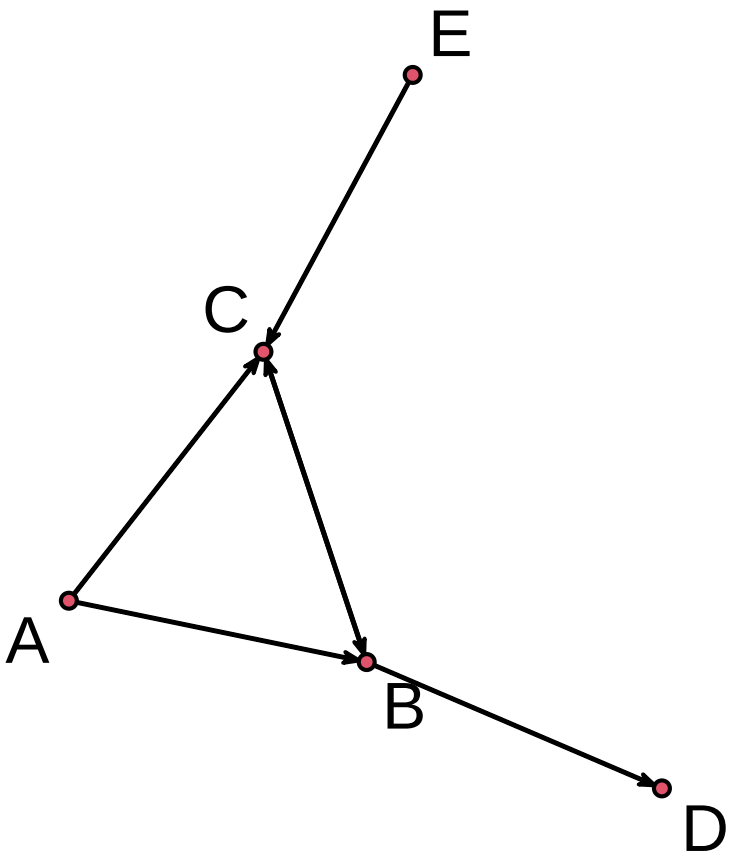
Closeness Centrality

You can calculate Closeness Centrality using sna package: need to assume that the graph is undirected

```
sna::closeness(net, gmode="graph")
```

```
## [1] 0.6666667 0.8000000 0.8000000 0.5000000 0.5000000
```

```
plot(net, displaylabels=T)
```



Use igraph

Specify the following arguments normalized=T, to normalize the measure mode="all" for directed graph

```
net_i<-intergraph::asIgraph(net)
```

```
igraph::closeness(net_i,  mode="all", normalized = T)
```

```
## [1] 0.6666667 0.8000000 0.8000000 0.5000000 0.5000000
```

Betweenness for the weighted graph

In igraph package, weights for the edges are considered as being "costs" rather than strengths. So if your weight variable is actually showing the strength, than you can take the inverse of it.

```
igraph::betweenness(net_i)
```

```
## [1] 0 3 2 0 0
```

```
sna::betweenness(net)
```

```
## [1] 0 3 2 0 0
```

Calculate closeness of the weighted graph with igraph package

```
library(intergraph)
network::set.edge.attribute(net, "weight", c(30,10,5,2,6))
igraph::closeness(asIgraph(net), normalized=T)
```

```
## [1] 0.04615385 0.06250000 0.04000000        NaN 0.04411765
```

```
network::set.edge.attribute(net, "weight", 1/c(30,10,5,2,6))
igraph::closeness(asIgraph(net), normalized = T)
```

```
## [1] 10.000000  3.750000  8.571429       NaN  4.090909
```

Betweenness centrality

Betweenness centrality measures the extent that a node sits 'between' pairs of other nodes in the network, such that a path between the other nodes has to go through that node.

A node with high betweenness is prominent, because that node is in a position to observe or control the flow of information in the network.

The equation for betweenness centrality is

$$C_b(n_i) = \sum g_{jk}(n_i)/g_{jk}$$

$g_{jk}(n_i)$ is the number of geodesic shortest paths between j and k nodes that contains node i
g_{jk} is the number of shortest geodesic paths between j and k.

Betweenness centrality

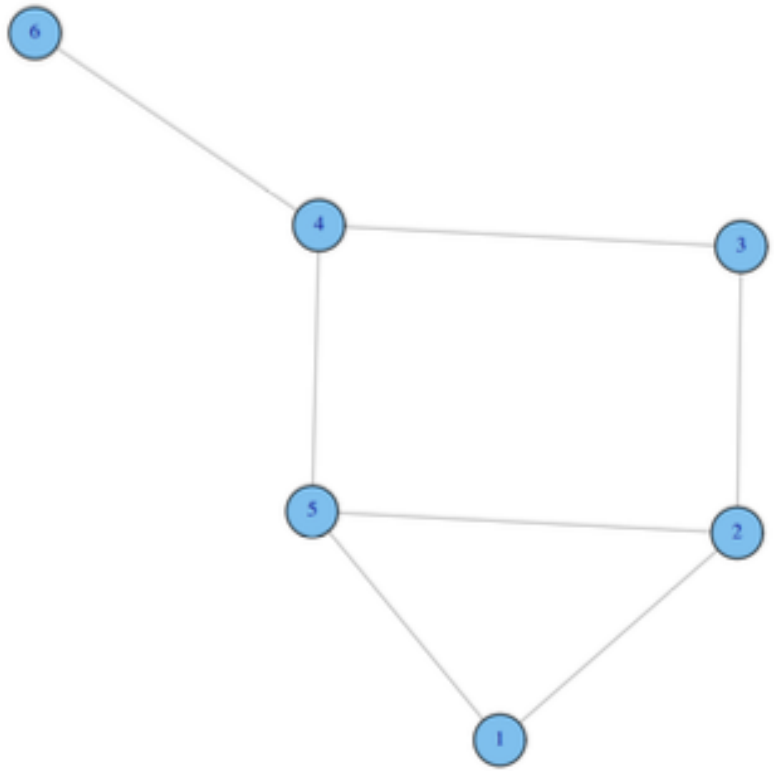
Betweenness centrality can be normalized by dividing the measure by the maximum possible value of it,

(n-1)(n-2) for directed graph (n-1)(n-2)/2 for non-directed graph

Betweenness centrality

Find the betweenness centrality for Node 2

It is an undirected graph, so total number of possible node pairs to compare: (n-1)*(n-2)/2



Betweenness centrality

For example for node 2, the (n-1)(n-2)/2 = 5(5-1)/2 = 10
terms in the summation in the order of 13,14,15,16,34,35,36,45,46,56 are

$1 + 0/1 + 0/1 + 0/1 + 0/1 + 0/1 + 1/2 + 0/1 + 0/1 + 0/1 + 0/1 = 1.5$

Betweenness centrality

Betwenness

```
igraph::betweenness(net_i)
```

```
## [1] 0 3 2 0 0
```

```
sna::betweenness(net)
```

```
## [1] 0 3 2 0 0
```

Closeness for weighted graph

- The best option to calculate closeness for the weighted network, is igraph package.
- If the edge in the igraph object has a "weight" attribute than it is used as a weight vector, otherwise need to specify in igraph::closeness function
- However: in igraph the weight of the edge is considered as being a cost rather than strength, example: weight is the time needed to get from one node to another
- Thus you can use the inverse (1/w) of the actual weights to get the closeness

Eigenvector centrality

- Eigenvector centrality takes into account not only the connectiveness of the vertex, but also to other vertices that are highly connected to others (prominent nodes)
- Higher value for a eigenvector centrality is showing higher connectiveness of the node

```
eigen_centrality(net_i)$vector
```

```
## [1] 0.6666667 1.0000000 1.0000000 0.3333333 0.3333333
```

Betweenness centrality

```
igraph::betweenness(asIgraph(net), normalized=F)
```

```
## [1] 0 3 2 0 0
```

eigenvector centrality

PageRank centrality

- PageRank (PR) is an algorithm used by Google Search to rank websites in their search engine results. PageRank was named after Larry Page, one of the founders of Google.
- PageRank is a way of measuring the importance of website pages.
- PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites
- If we consider a node as being a webpage and a edge as a hyperlink from one webpage to another, we can use PageRank a s anode importance metric.

PageRank centrality

```
page_rank(net_i)$vector
```

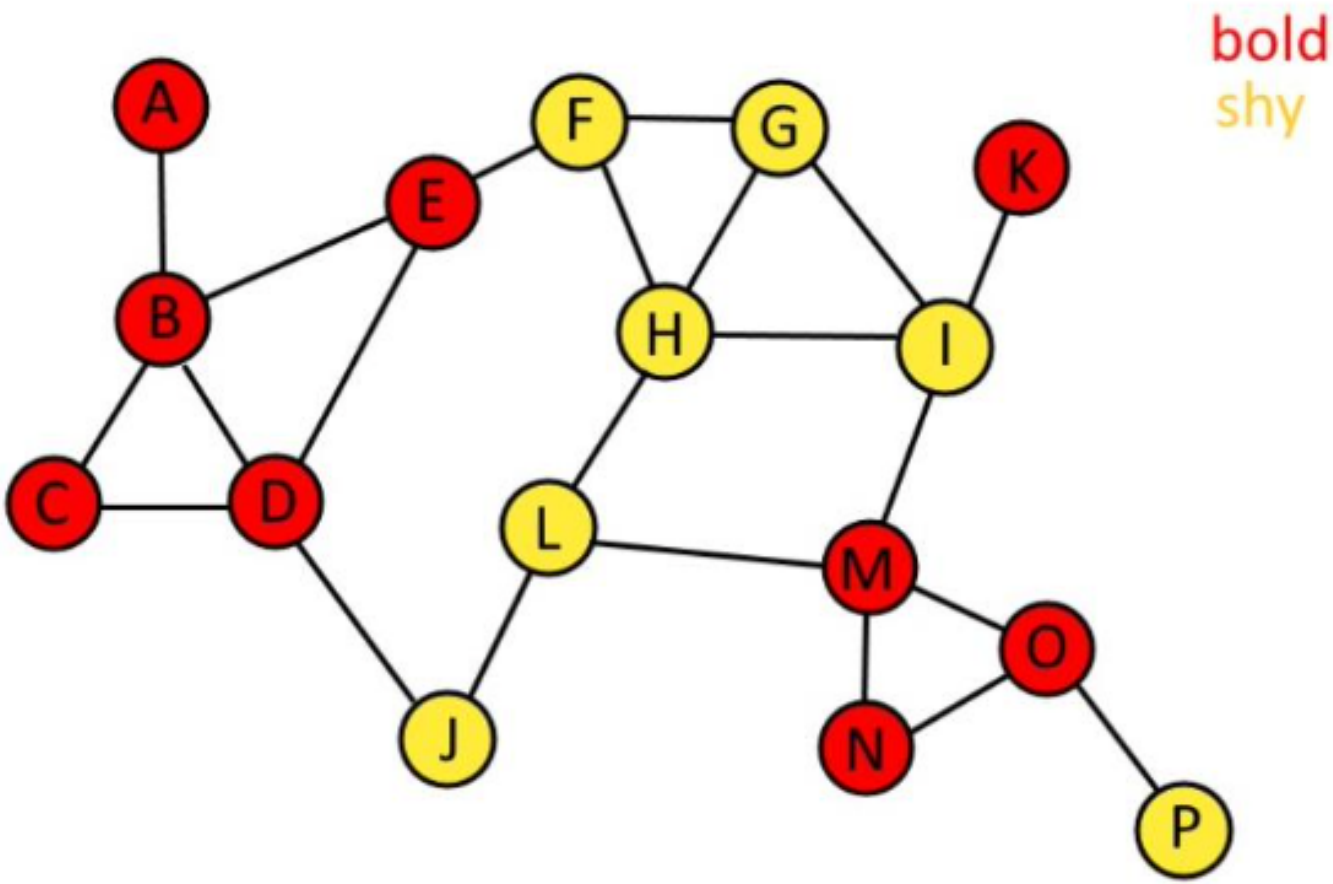
```
## [1] 0.06665384 0.35048702 0.30059447 0.21561083 0.06665384
```

Special relations in networks

Special relations in networks

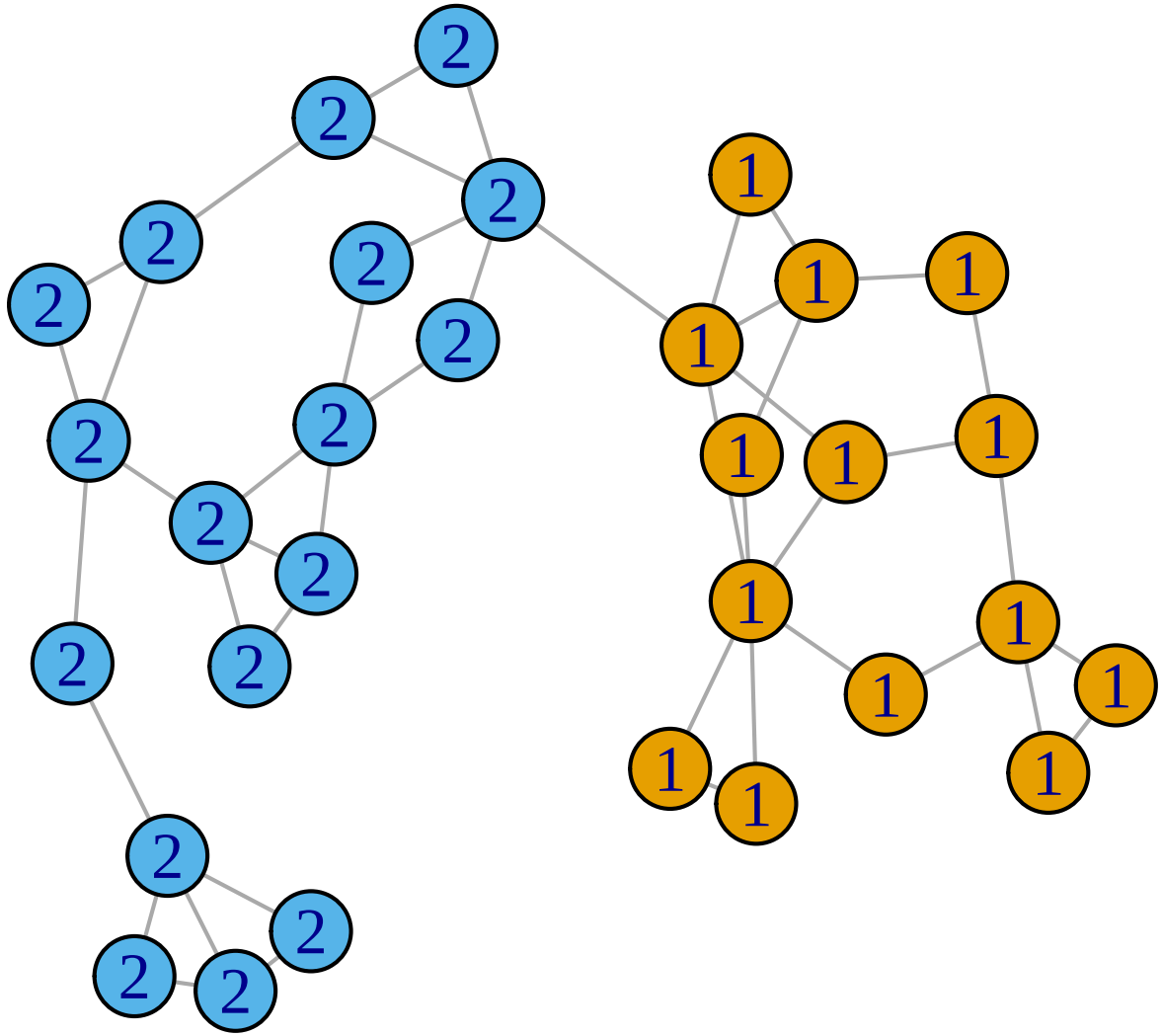
Special relations in networks

Assortativity, is a preference for a network's nodes to attach to others that are similar in some way. Or, how likely is that two networks sharing same attribute together, will be connected



Special relations in networks

```
set.seed(1)
plot(Moreno_i, vertex.color = V(Moreno_i)$gender,
     vertex.label=V(Moreno_i)$gender)
```



Special relations in networks

Assortativity coefficient is a value between -1 and 1.

It is positive if nodes with the same attributes tend to be connected to each other (**homophily**).

The opposite is when nodes with different characteristic tends to be connected to each other (**heterophily**)

```
assortativity(Moreno_i, V(Moreno_i)$gender)
```

```
## [1] 0.956336
```

We have very strong homophilic network

Special relations in networks

Can we have such an high assortativity by chance?

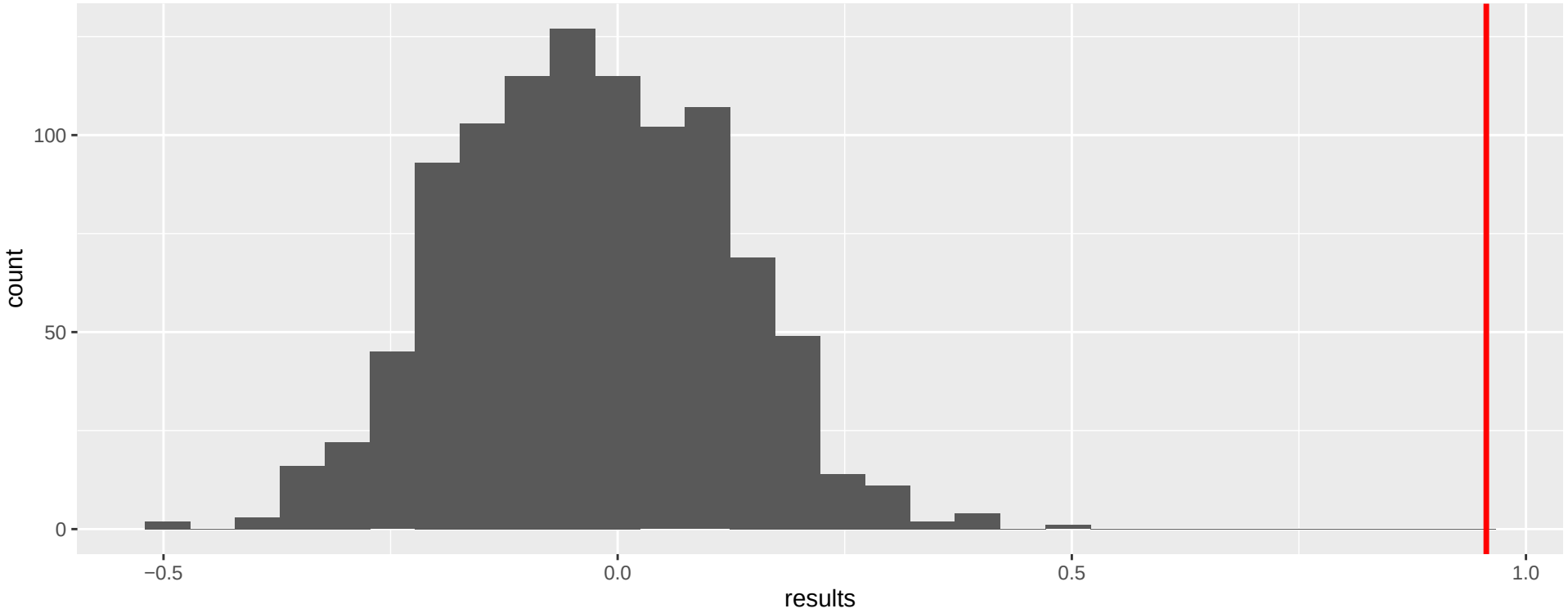
Lets calculate graphs assortavitiy by randomizing the node attribute gender

```
actual <- assortativity(Moreno_i, V(Moreno_i)$gender)

results <- vector(length=1000)
for(i in 1:1000){
  results[i] <- assortativity(Moreno_i,
      sample(V(Moreno_i)$gender))
}
```

Special relations in networks

```
ggplot()+geom_histogram(aes(x=results))+
  geom_vline(xintercept = actual,
             size=1.2, color="red")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```




```
sum(results>actual)
```

```
## [1] 0
```

Special relations in networks

Degree assortativity is the tendency for nodes of high degree (low degree) in a graph to be connected to high degree nodes (low degree).

```
assortativity.degree(Moreno_i, directed=F)
```

```
## [1] -0.234632
```

Negative value indicates that highly connected nodes don't connect to other highly connected nodes

Special relations in networks

In network science, **reciprocity** is a measure of the likelihood of vertices in a directed network to be mutually linked.

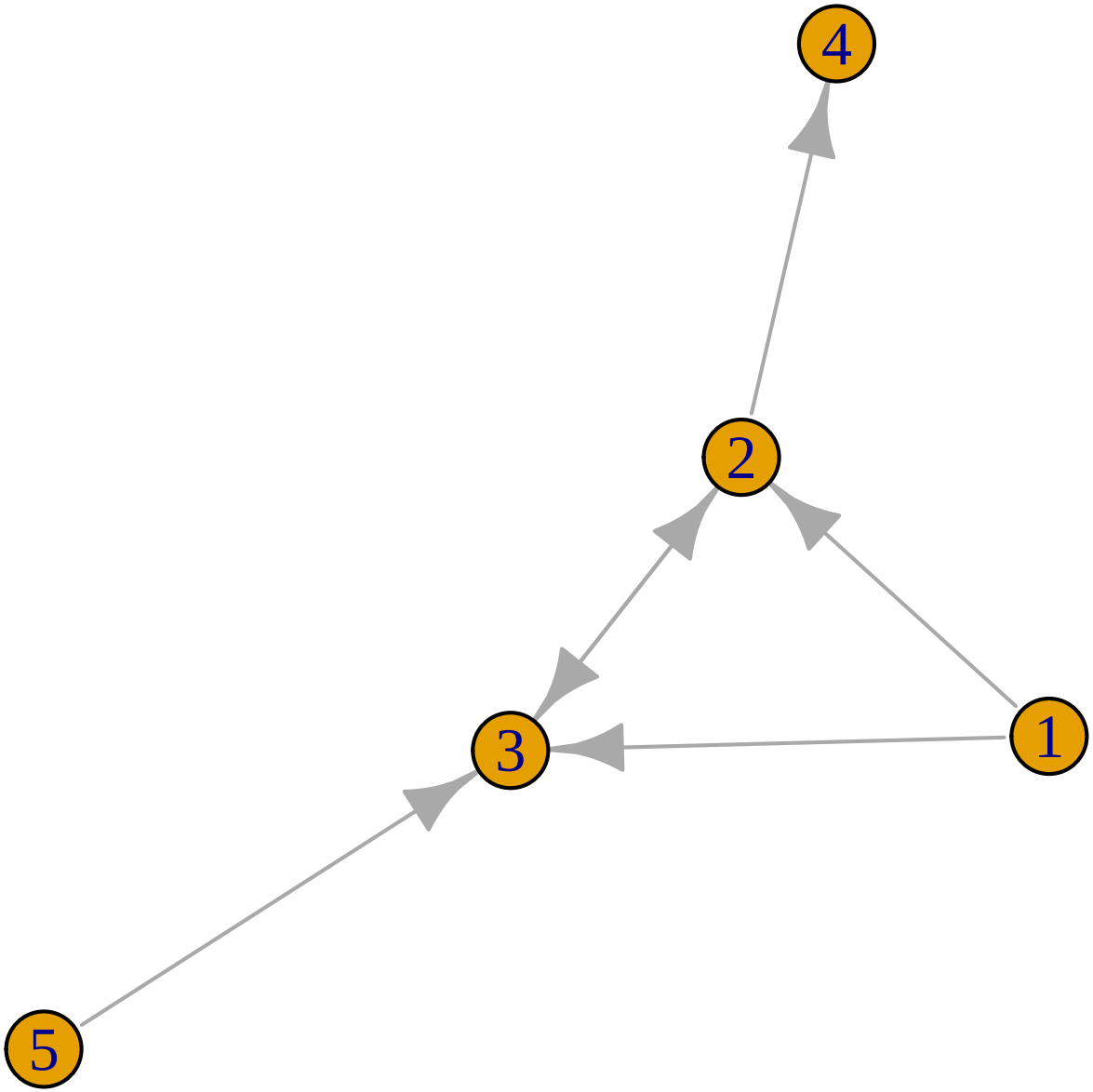
$$r = \frac{L^{<->}}{L}$$

Where L is the total number of edges and $L^{<->}$ is the number of edges pointing in both directions.

r = 0, for a purely undirectional network, while r = 1, for a purely directional network.

Special relations in networks

```
plot(net_i, displayLabels=T)
```



Special relations in networks

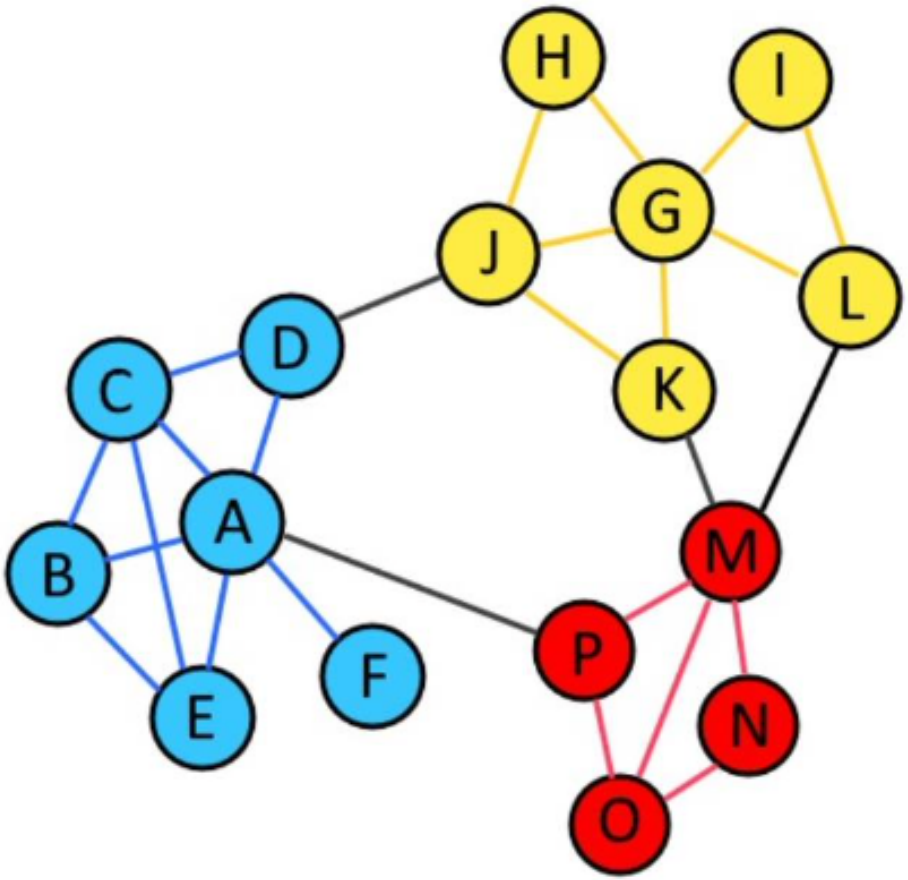
Calculate reciprocity

```
reciprocity(net_i)
```

```
## [1] 0.3333333
```

Community detection

If the network has a community structure, then it is possible to assign vertices to unique sets, called communities, groups, clusters. Within each community, edge density is higher, thus nodes are more connected.



Community

Community detection:

- Find vertices that are more connected to each other than the others.
- Community detection can be very computing incentive especially for large networks.

Community detection

Community detection with Fast greedy algorithm

```
com <- fastgreedy.community(Moreno_i)
sizes(com)
```

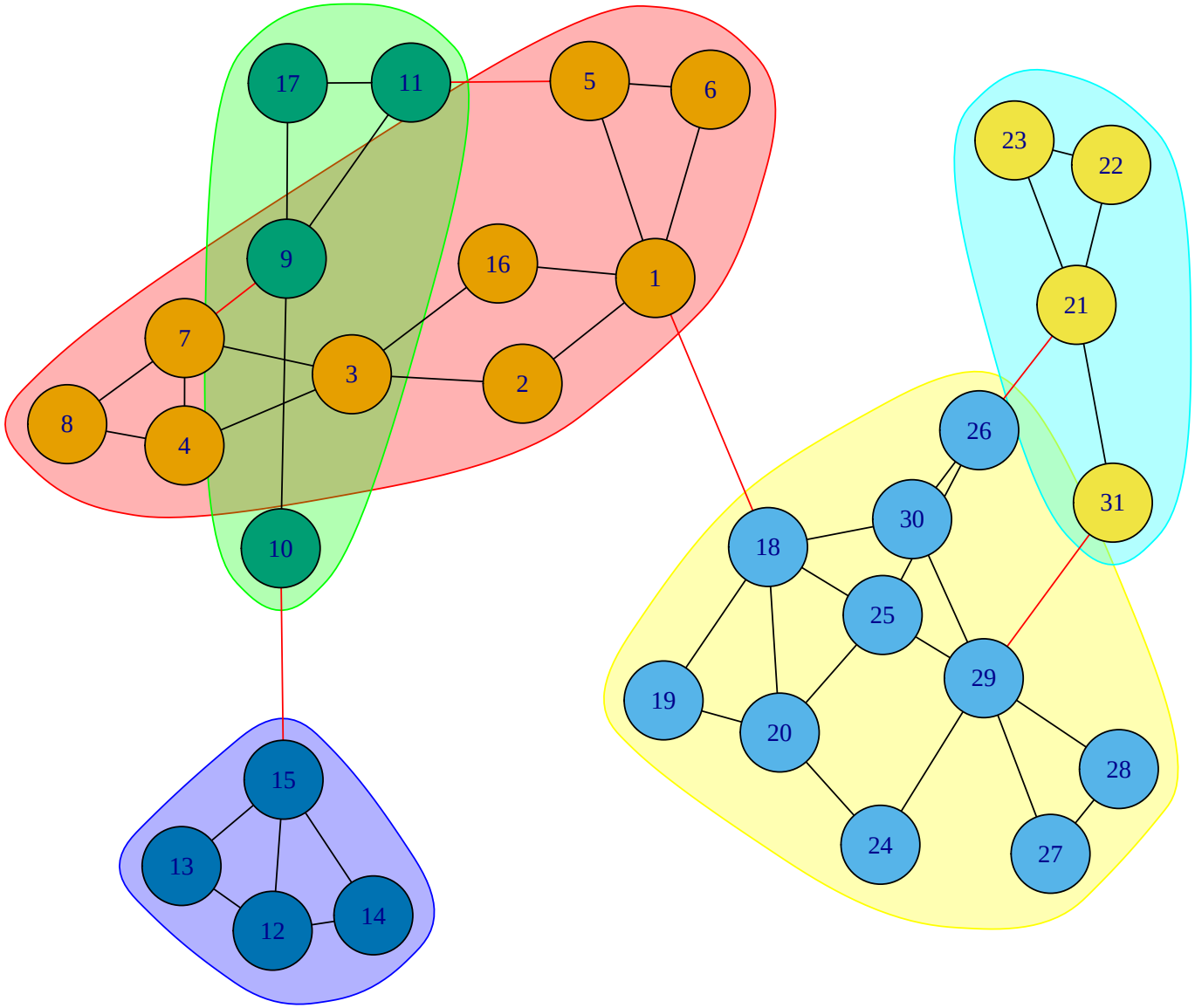
```
## Community sizes
##  1  2  3  4  5  6
##  9 10  4  4  4  2
```

```
membership(com)
```

```
##  [1] 1 1 1 1 1 1 1 1 3 3 3 5 5 5 5 1 3 2 2 2 4 4 4 2 2 2 2 2 2 2 4 6 6
```

Community detection

```
plot(com, Moreno_i)
```



Community detection

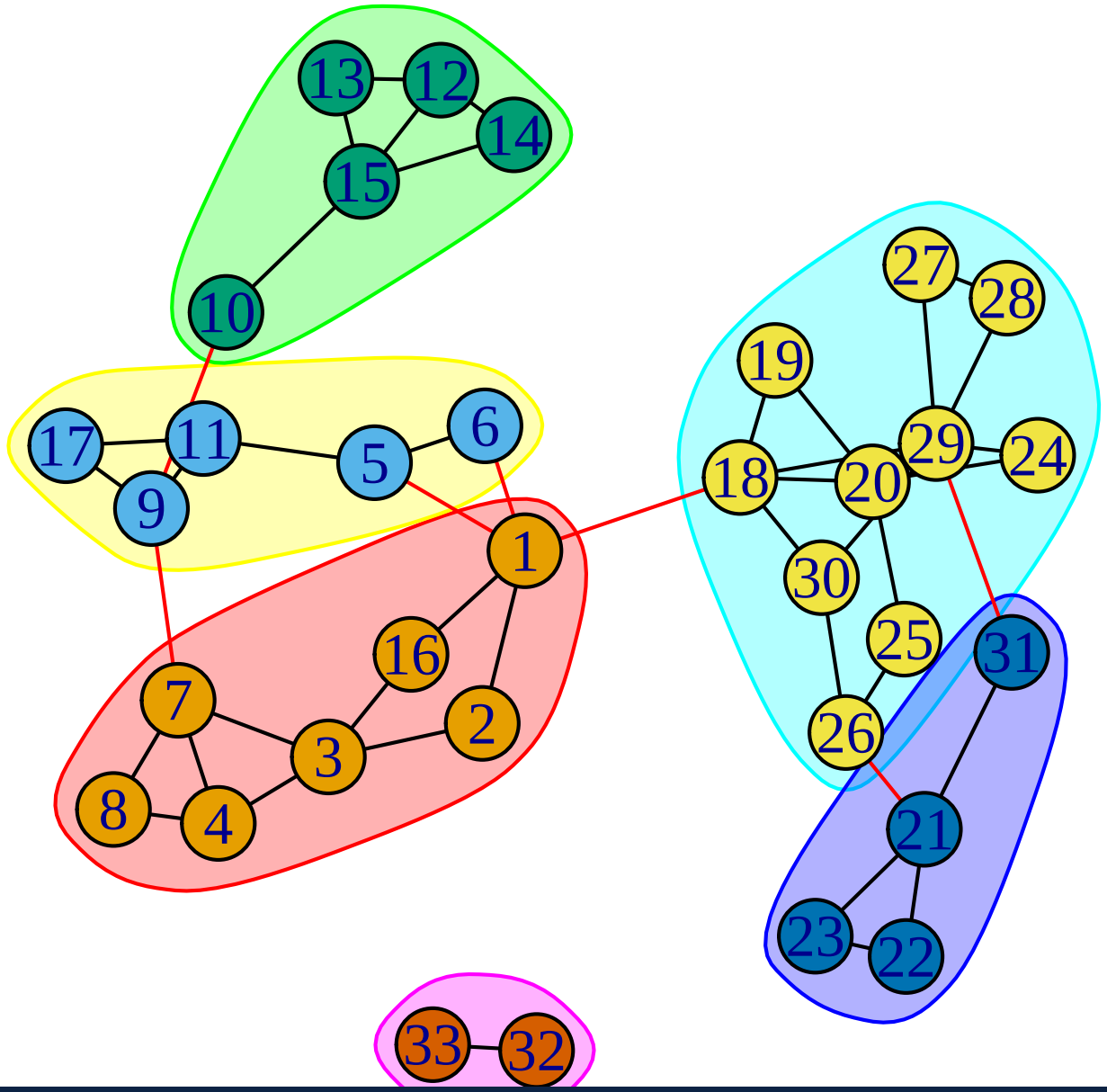
Edge-betweenness community detection

```
edge_b <- edge.betweenness.community(Moreno_i)
sizes(edge_b)
```

```
## Community sizes
##  1  2  3  4  5  6
##  7  5  5 10  4  2
```

Community detection

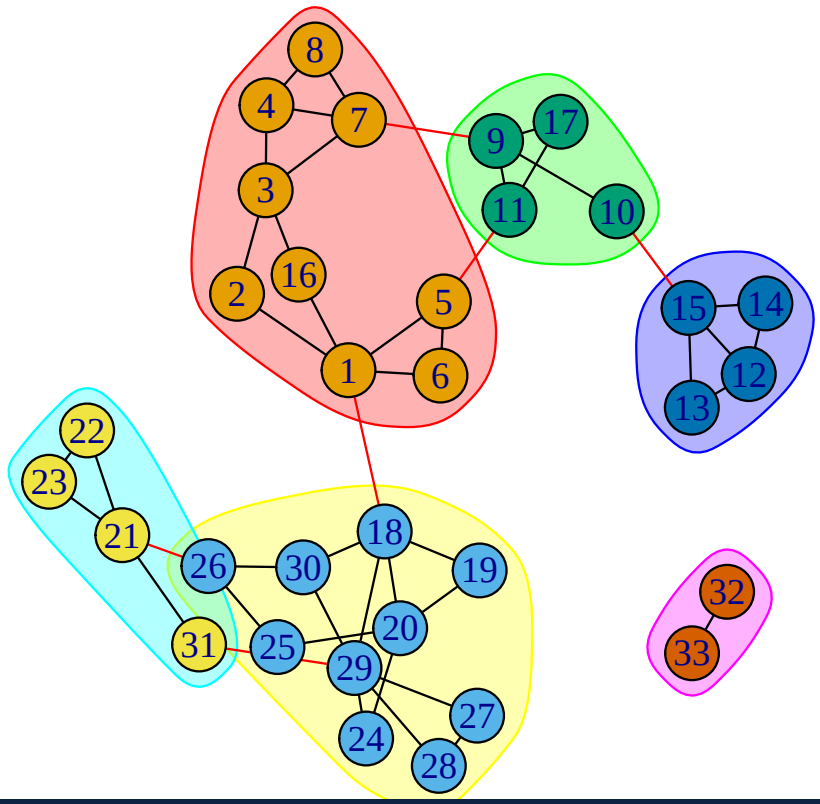
```
plot(edge_b, Moreno_i)
```



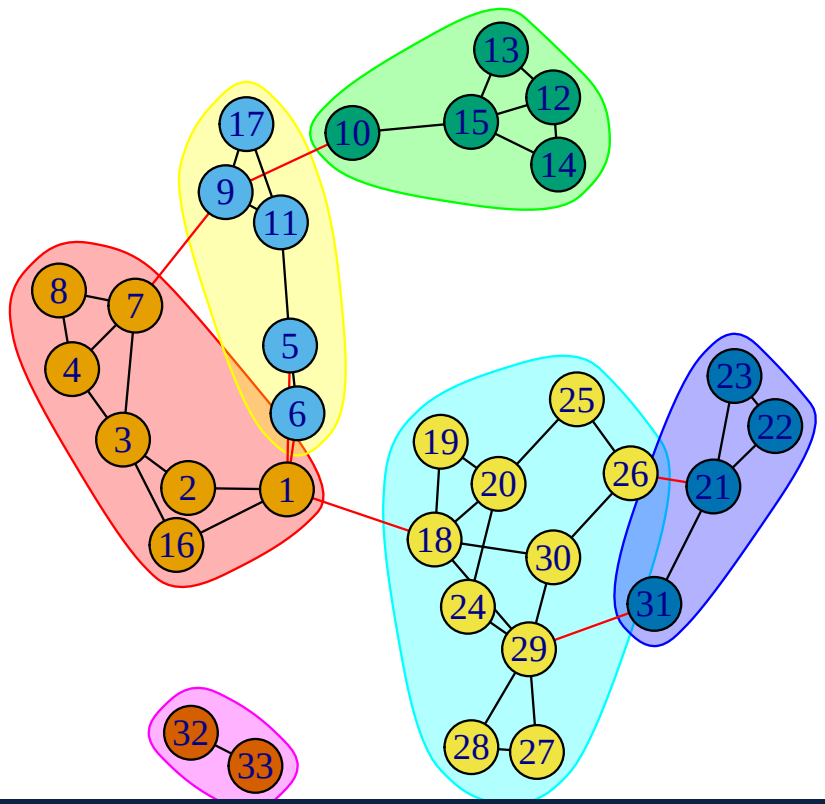
Community detection

```
par(mfrow = c(1, 2))
plot(com, Moreno_i, main="Fast greedy")
plot(edge_b, Moreno_i, main="Edge Betweenness")
```

Fast greedy



Edge Betweenness



A k-core is a maximal subgraph where each vertex is connected to at least k other vertices in the subgraph. k-cores have a number of advantages: they are nested (every member of a 4-core is also a member of a 3-core, and so on), they do not overlap, and they are easy to identify.

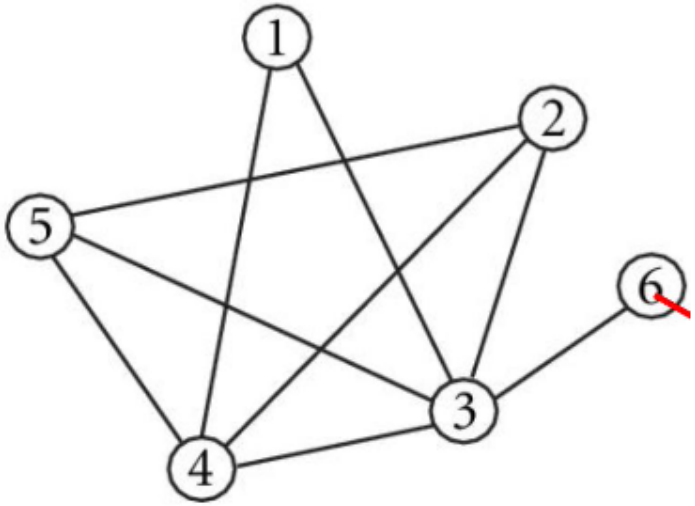
To identify the k-core structure in the network, the coreness function is used. It returns a vector listing the highest core that each vertex belongs to in the network.

```
coreness(Moreno_i)
```

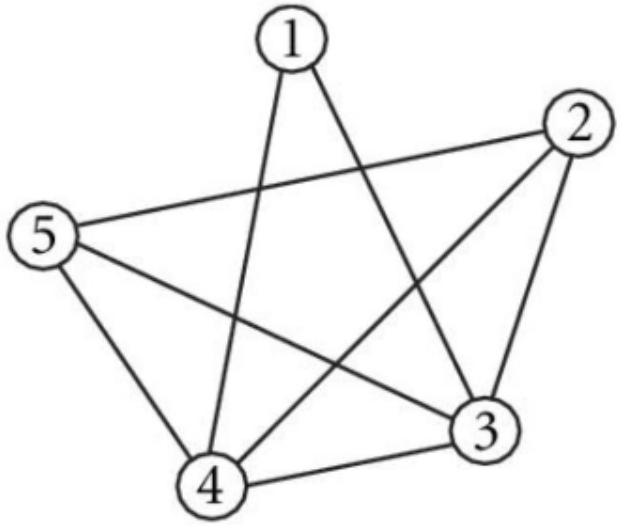
```
##  [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1
```

K-cores

A k-core of a graph is a maximal subgraph such that each node in the subgraph has at least degree k



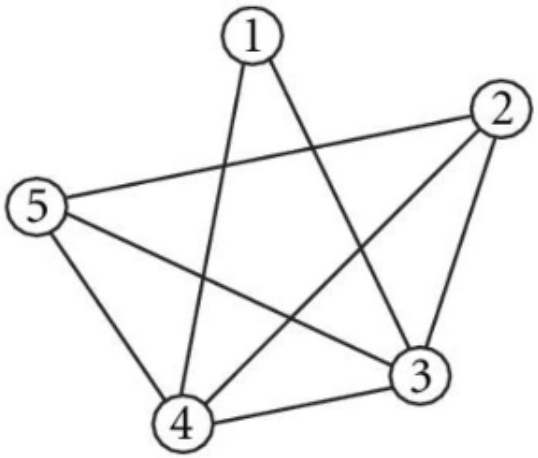
Two cores



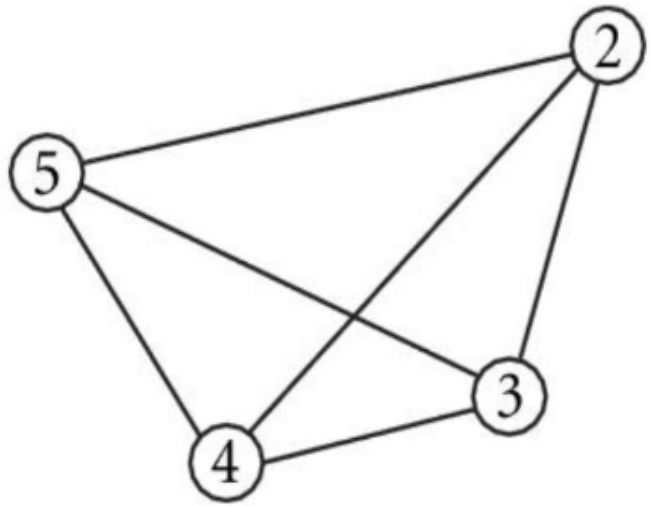
K-cores

A k-core of a graph is a maximal subgraph such that each node in the subgraph has at least degree k

Two cores



Three (maximal cores)



US air flights network

US air flights network

US air flights network

- Network and igraph objects
- Describing USA air flights network
- edge attribute weight is showing the number of flights

```
load("./data/usa.rda")
load("./data/usa_i.rda")
network::list.vertex.attributes(usa)
```

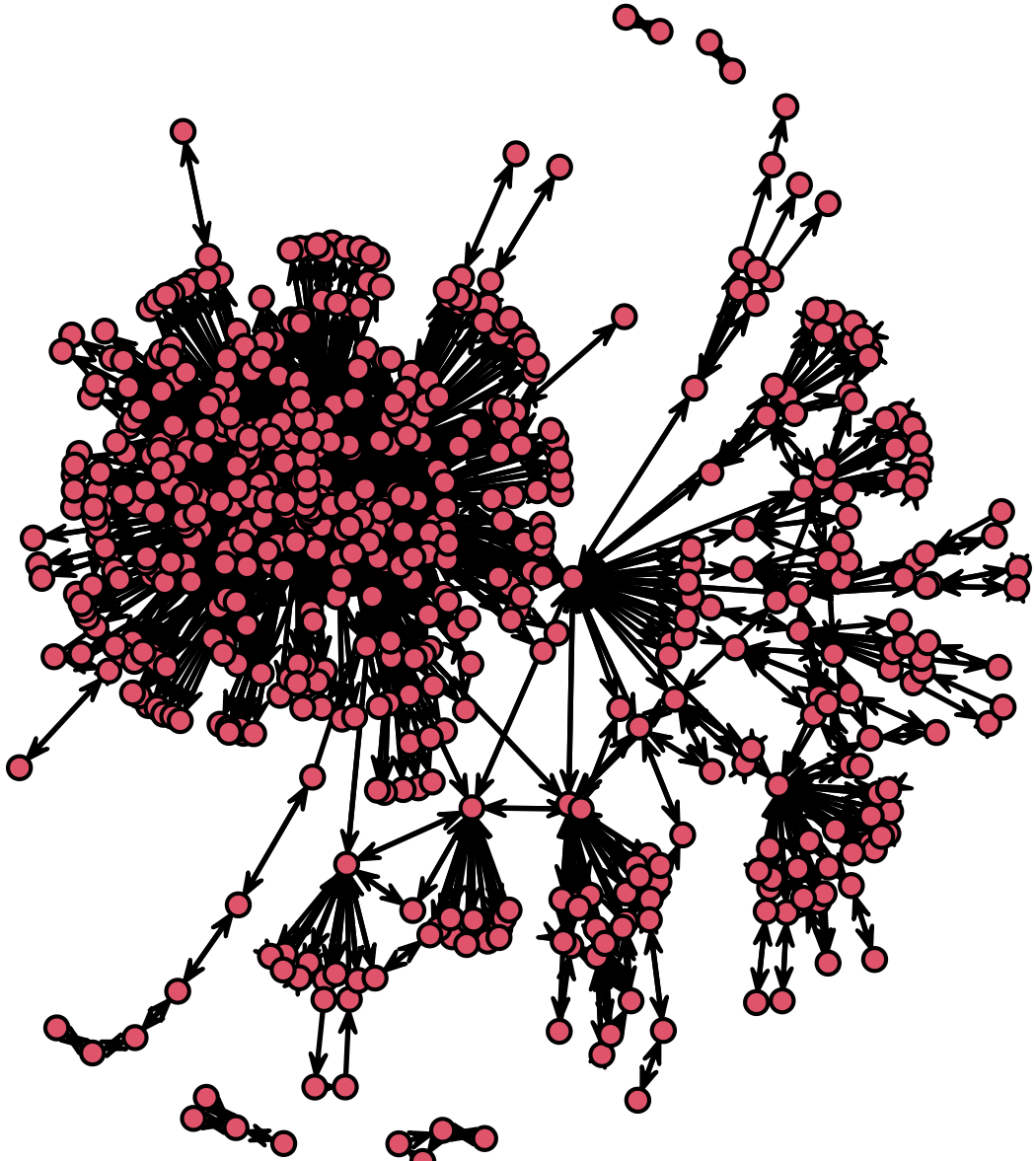
```
## [1] "country"      "lat"          "long"         "na"           "type"
## [6] "vertex.names"
```

```
network::list.edge.attributes(usa)
```

```
## [1] "na"     "weight"
```

US air flights network

```
plot(usa)
```



US air flights network

Calculating network statistics

diameter

```
get_diameter(usa_i)
```

```
## + 13/610 vertices, named, from 9e3c46e:
##  [1] LPS WSX DHB RCE FBS LKE SEA JNU ANC BET OOK TNK WWT
```

edge density

```
edge_density(usa_i)
```

```
## [1] 0.01505828
```

mean distance (average path length)

```
mean_distance(usa_i, directed = T)
```

```
## [1] 4.800206
```

Assortativity

To calculate assortativity, first transform the node attribute into numeric vectore

```
list.vertex.attributes(usa_i)
```

```
## [1] "country" "lat"     "long"    "na"      "type"    "name"
```

```
table(V(usa_i)$type)
```

```
##
##         closed  large_airport medium_airport  seaplane_base  small_airport
##              2            134            318             26            130
```

```
type_num <- as.numeric(as.factor(V(usa_i)$type))
assortativity(usa_i, type_num)
```

```
## [1] 0.3529708
```

Assortativity and Reciprocity

```
assortativity.degree(usa_i, directed=T)
```

```
## [1] -0.3146973
```

```
reciprocity(usa_i)
```

```
## [1] 0.9678227
```

US air flights network

How many airports can you reach from Ney York's JFK directly ?

```
ego(usa_i,1, "JFK", mode=c("out"))
```

```
## [[1]]
## + 58/610 vertices, named, from 9e3c46e:
##  [1] JFK ABQ ATL AUS BNA BOS BTV BUF BUR BWI CHS CLE CLT CMH CVG DCA DEN DFW DTW
## [20] FLL HNL HOU IAD IND JAX LAS LAX LGB MCO MIA MSP MSY OAK ORD ORF PBI PDX PHL
## [39] PHX PIT PWM RDU RIC ROC RSW SAN SAT SAV SDF SEA SFO SJC SLC SMF SRQ STL SYR
## [58] TPA
```

US air flights network

How many airports can you reach from Ney York's JFK directly ?

```
ego(usa_i,2, "JFK", mode=c("out"))
```

```
## [[1]]
## + 386/610 vertices, named, from 9e3c46e:
##   [1] JFK ABQ ATL AUS BNA BOS BTV BUF BUR BWI CHS CLE CLT CMH CVG DCA DEN DFW
##  [19] DTW FLL HNL HOU IAD IND JAX LAS LAX LGB MCO MIA MSP MSY OAK ORD ORF PBI
##  [37] PDX PHL PHX PIT PWM RDU RIC ROC RSW SAN SAT SAV SDF SEA SFO SJC SLC SMF
##  [55] SRQ STL SYR TPA CNM DAL IAH LAM MCI MDW ABE ABY AEX AGS ALB ATW AVL AVP
##  [73] AZO BDL BHM BMI BQK BTR CAE CAK CHA CHO CID COS CRW CSG DAB DAY DHN DSM
##  [91] ECP ELP EVV EWN EWR EYW FAR FAY FNT FSD FSM FWA GNV GPT GRB GRK GRR GSO
## [109] GSP GTR HPN HSV ICT ILM JAN LAN LEX LFT LGA LIT LWB MBS MCN MDT MEI MEM
## [127] MGM MHT MKE MLB MLI MLU MOB MSL MSN MYR OAJ OKC OMA PHF PIA PIB PNS PVD
## [145] ROA SBN SGF SHV SNA TLH TRI TTN TUL TUP TUS TYS VLD VPS XNA HRL LBB AHN
## [163] MKL ACK ACY AUG BHB HYA ISP LEB MVY PBG PQI PVC RKD RUT SLK SFB IPL BFD
## + ... omitted several vertices
```

```
ego_size(usa_i,2, "JFK", mode=c("out"))
```

```
## [1] 386
```

US air flights network

What are the airports that have direct flights TO JFK

```
neighbors(usa_i, "JFK", mode="in")
```

```
## + 57/610 vertices, named, from 9e3c46e:
##  [1] ABQ ATL AUS BNA BOS BTV BUF BUR BWI CHS CLE CLT CMH CVG DCA DEN DFW DTW FLL
## [20] HNL HOU IAD IND JAX LAS LAX LGB MCO MIA MSP MSY OAK ORD ORF PBI PDX PHL PHX
## [39] PIT PWM RDU RIC ROC RSW SAN SAT SAV SDF SEA SFO SJC SLC SMF SRQ STL SYR TPA
```

US air flights network

Find the degree centrality

```
d_out <- igraph::degree(usa_i, mode = "out", normalized = T)
which.max(d_out)
```

```
## ATL
##  41
```

```
d_out[41]
```

```
##       ATL
## 0.2512315
```

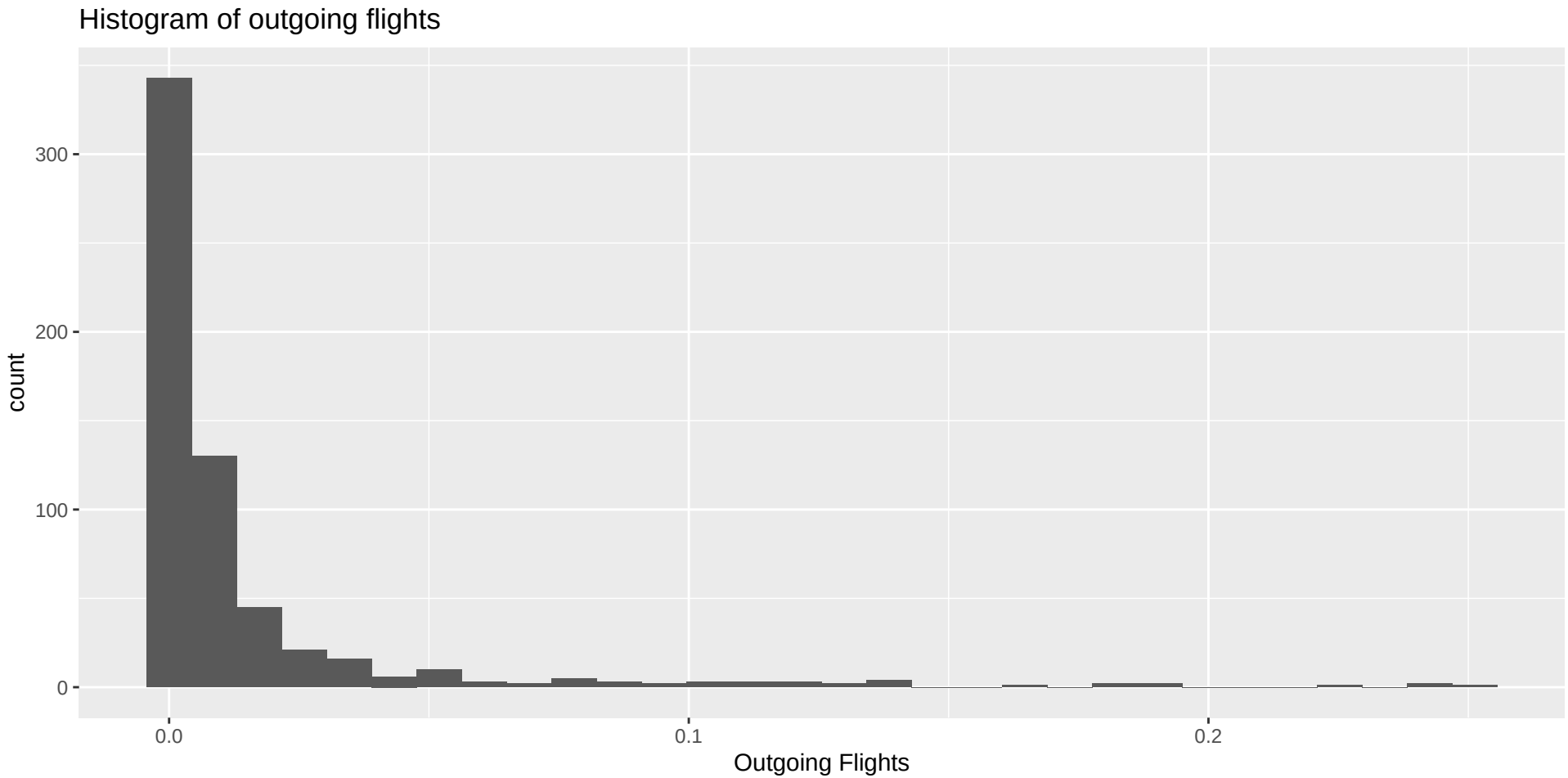
ATL- Atlanta has the highest number of outgoing connections

You can reach 153 airports from Atlanta

US air flights network

```
ggplot()+geom_histogram(aes(x=d_out))+
  labs(x="Outgoing Flights",
       title="Histogram of outgoing flights")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Us air flights network

Normalized version of degree centrality

```
d_in <- igraph::degree(usa_i, mode = "in", normalized = T)
which.max(d_in)
```

```
## ATL
##  41
```

```
d_in[41]
```

```
##       ATL
## 0.2495895
```

US air flights network

How many flights? * As an edge weight, the edge attribute “weight” is used * In order for igraph to recognize the weights, the attribute needs to be named exactly “weight

```
f_in <- strength(usa_i, mode = "in")
which.max(f_in)
```

```
## ATL
##  41
```

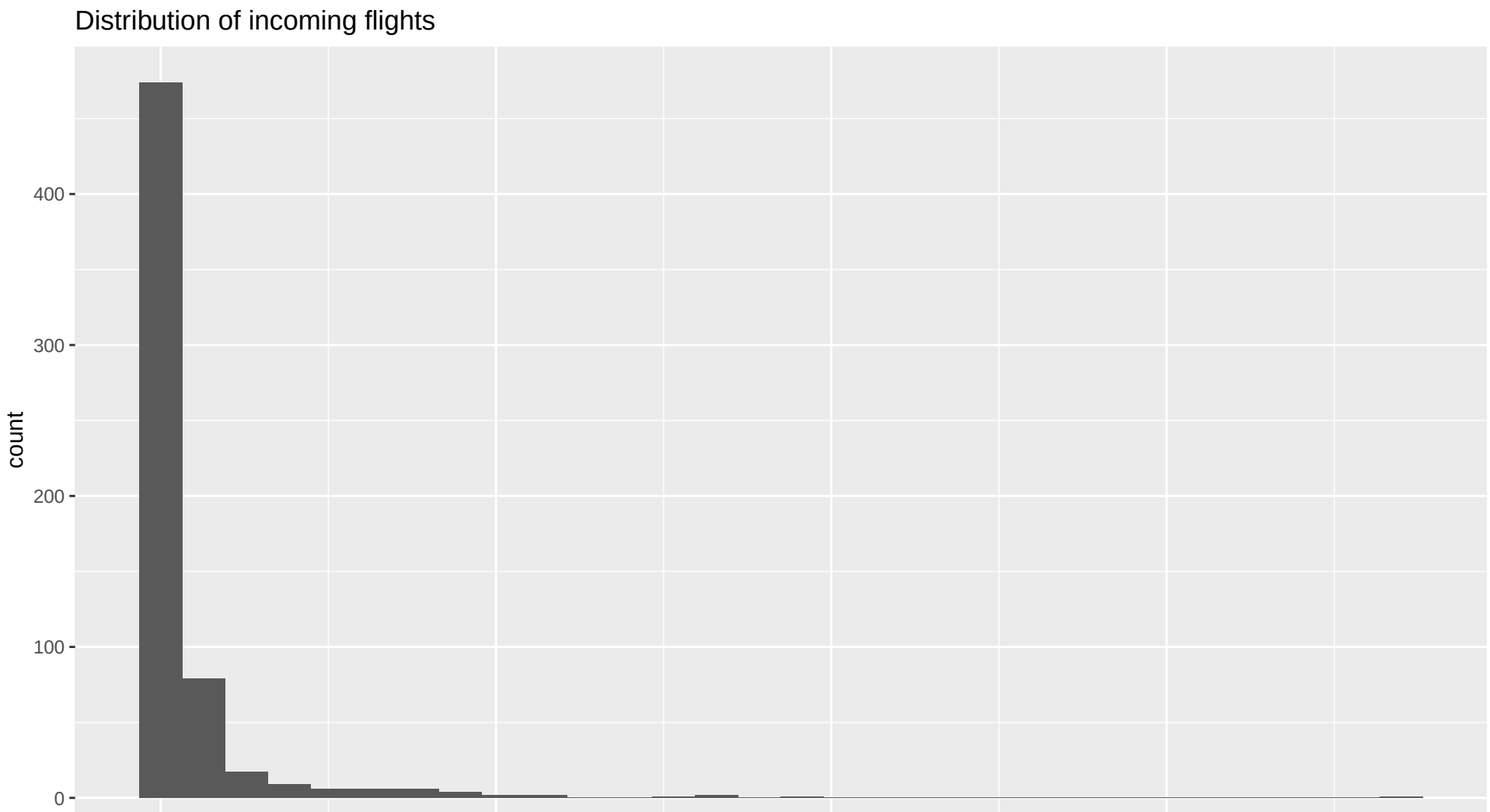
```
f_in[41]
```

```
## ATL
## 741
```

US air flights network

```
ggplot()+geom_histogram(aes(x=f_in))+
  labs(x="Flights", title="Distribution of incoming flights")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



US air flights network

Closeness:
weight is considered as a cost, so we take its inverse.

```
close <- igraph::closeness(usa_i,weights = 1/E(usa_i)$weight,
                           normalize=T, mode="all")
df_c <- data.frame(airports=names(close),closeness=close)
df_c <- df_c[order(df_c$closeness, decreasing=T),]
head(df_c)
```

```
##     airports closeness
## BFI      BFI         1
## BLD      BLD         1
## CKX      CKX         1
## DUT      DUT         1
## GCW      GCW         1
## TKJ      TKJ         1
```

US air flights network

Betweenness

```
bet <- igraph::betweenness(usa_i, weights = 1/E(usa_i)$weight,
                           normalize=T)
sort(bet, decreasing = T)[1:5]
```

```
##        ATL        ANC        SEA        DEN        BET
## 0.59747853 0.39785125 0.36194835 0.14779783 0.09277954
```

US air flights network

For page rank calculations, weight is considered as a strength

```
p_rank <- page_rank(usa_i)$vector
sort(p_rank, decreasing = T)
```

```
##          ATL          DEN          ORD          DFW          LAX          ANC
## 0.0440659321 0.0249241294 0.0241450742 0.0221453566 0.0193948277 0.0153221617
##          CLT          PHX          MSP          LAS          BET          PHL
## 0.0143345586 0.0138800351 0.0131525171 0.0129836305 0.0127407020 0.0122473009
##          SEA          DTW          DCA          SLC          BOS          FAI
## 0.0122021004 0.0107022553 0.0099982158 0.0099761133 0.0099180401 0.0098469786
##          SFO          MCO          JFK          IAH          JNU          LGA
## 0.0096609760 0.0096357929 0.0093479090 0.0089962597 0.0076682880 0.0076384150
##          BWI          IAD          MDW          PDX          STL          OTZ
## 0.0075597334 0.0072778879 0.0072023380 0.0071998475 0.0070580801 0.0069412372
##          TPA          KTN          MIA          MSY          FLL          OME
## 0.0068543925 0.0068377140 0.0067707238 0.0066781985 0.0065020115 0.0064791549
##          EWR          SAN          CLE          ANI          BNA          MCI
## 0.0061758634 0.0061085266 0.0050881991 0.0050052943 0.0047602011 0.0045051755
##          HNL          AKN          AUS          SFB          RDU          HOU
## 0.0045006944 0.0044456880 0.0043361574 0.0042905864 0.0040863554 0.0040264562
##          CVG          PIT          IND          OAK          CMH          RSW
## 0.0039865616 0.0036863489 0.0036503480 0.0035948290 0.0034572221 0.0034497024
##          MEM          MKE          SAT          AZA          SJC          SMF
## 0.0033885817 0.0033268189 0.0032773308 0.0032744458 0.0032467233 0.0032173280
```

US air flights network

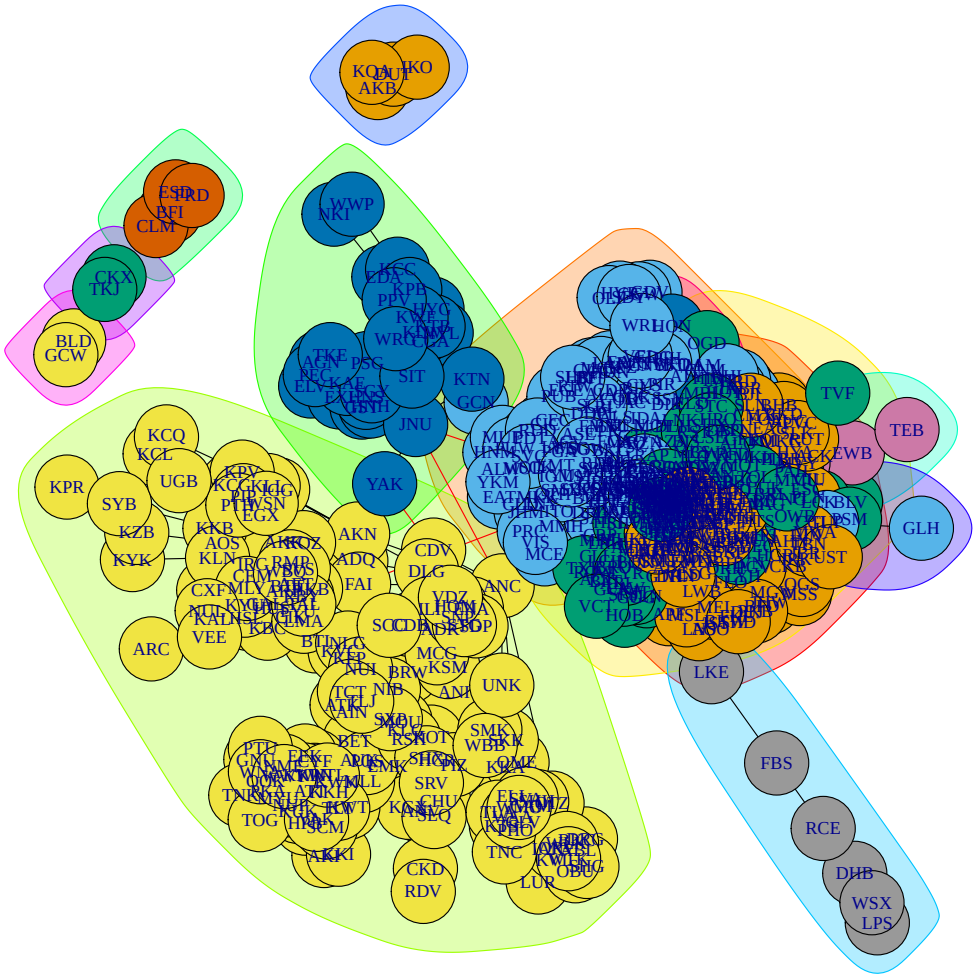
- Community detection, using fastgreedy algorithm
- You need to transform the graph into undirected first

```
usa_i1 <- as.undirected(usa_i, mode = "collapse")
com <- fastgreedy.community(usa_i1)
sizes(com)
```

```
## Community sizes
##   1   2   3   4   5   6   7   8   9  10  11  12  13
## 155 144 104 151  28   4   5   6   4   2   2   2   3
```

US air flights network

```
plot(com, usa_i1)
```



NBA passing

NBA passing

NBA passing

Load n_pass, Golden State Warriors passing network

```
load("./data/n_pass.rda")
i_pass <- asIgraph(n_pass)
igraph::list.edge.attributes(i_pass)
```

```
## [1] "FGP"    "na"     "passes"
```

```
igraph::list.vertex.attributes(i_pass)
```

```
## [1] "na"           "ngames"       "position"     "vertex.names"
```

NBA passing

- Set edge attribute (weight)
- Set vertex attribute (name)

```
i_pass <- set.edge.attribute(i_pass, "weight", value = E(i_pass)$passes)
i_pass <- set.vertex.attribute(i_pass, "name", value=V(i_pass)$vertex.names)
```

NBA passing

```
edge_density(i_pass)
```

```
## [1] 0.9571429
```

```
mean_distance(i_pass, directed=T)
```

```
## [1] 14.99048
```

NBA passing

```
pos_num <- as.numeric(as.factor(V(i_pass)$position))
assortativity(i_pass, pos_num)
```

```
## [1] -0.1298526
```

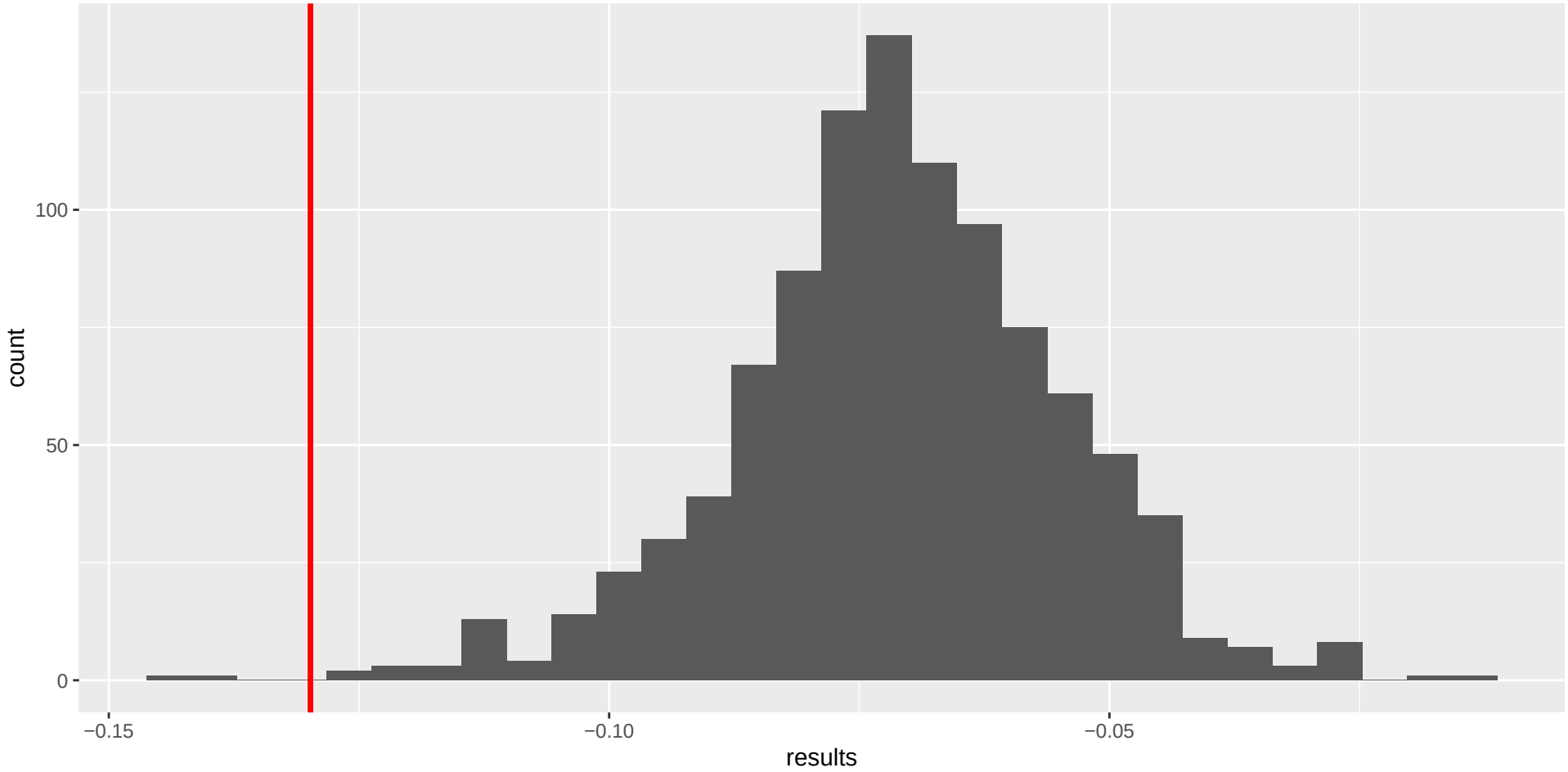
NBA passing

```
actual <- assortativity(i_pass, pos_num)

results <- vector(length=1000)
for(i in 1:1000){
  results[i] <- assortativity(i_pass,sample(pos_num))
}
```

NBA passing

```
ggplot()+geom_histogram(aes(x=results))+
  geom_vline(xintercept = actual,
             size=1.2, color="red")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



NBA passing

Degree assortativity

```
assortativity.degree(i_pass)
```

```
## [1] -0.1968207
```

NBA passing

For sure we are going to have high reciprocity value

```
reciprocity(i_pass)
```

```
## [1] 0.9850746
```

NBA passing

Now calculate node level statistics Passes recieved
Passes received

```
recieved <- strength(i_pass, mode="in")
recieved
```

```
##    Patrick McCaw      Jordan Bell     Damian Jones       Quinn Cook
##             1101              843               66             1334
##     Kevon Looney   Draymond Green    Klay Thompson    Stephen Curry
##              777             3446             2847             2993
##     JaVale McGee       Nick Young     Kevin Durant   Andre Iguodala
##              470             1197             3302             1916
## Shaun Livingston    Zaza Pachulia       David West
##             1740             1017             1323
```

NBA passing

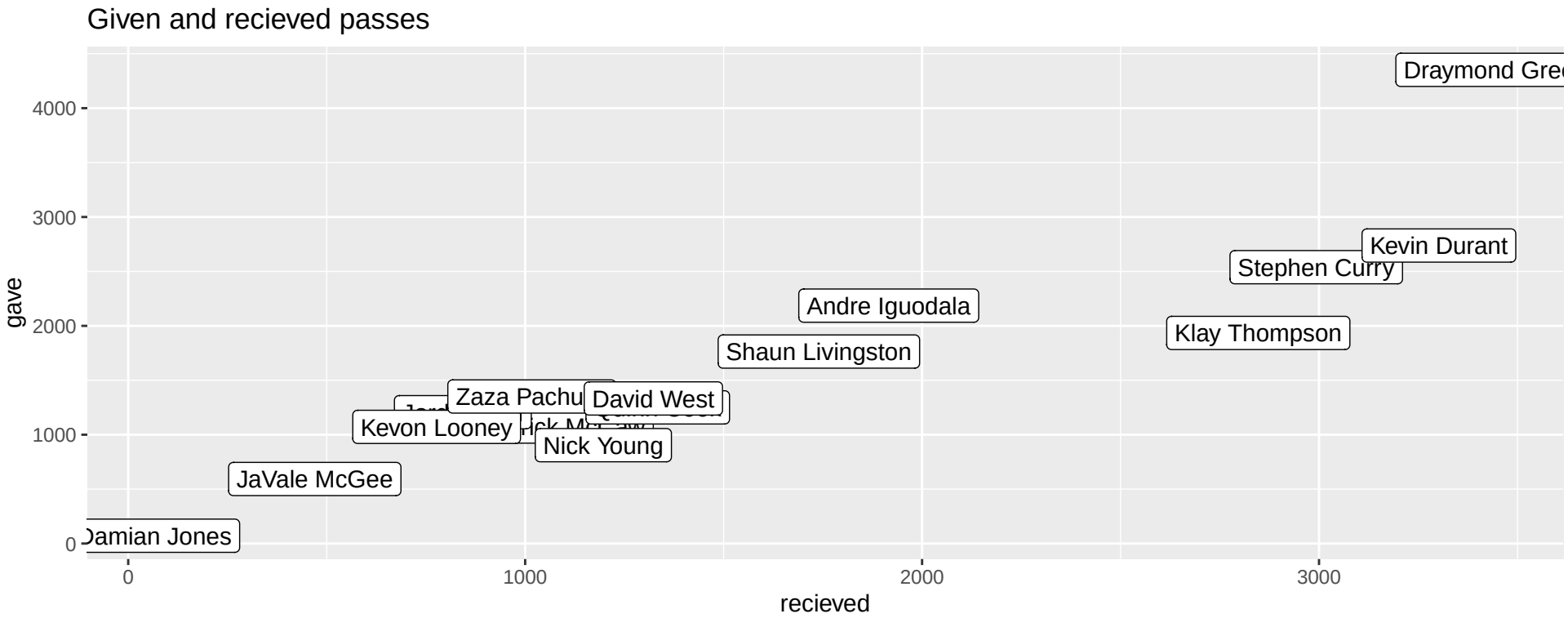
passes given

```
gave <- strength(i_pass, mode="out")
gave
```

```
##    Patrick McCaw      Jordan Bell     Damian Jones       Quinn Cook
##             1075             1207               72             1251
##     Kevon Looney   Draymond Green    Klay Thompson    Stephen Curry
##             1072             4352             1935             2539
##     JaVale McGee       Nick Young    Kevin Durant   Andre Iguodala
##              593              904             2738             2186
## Shaun Livingston    Zaza Pachulia      David West
##             1764             1351             1333
```

NBA passing

```
df_p <- data.frame(gave, recieved, players=names(recieved))
ggplot(df_p, aes(recieved, gave, label=players))+
  geom_point()+geom_label()+
  ggtitle("Given and recieved passes")
```



NBA passing

Degree centrality

```
degree(i_pass)
```

```
##    Patrick McCaw      Jordan Bell     Damian Jones       Quinn Cook
##               28               28               22               28
##     Kevon Looney   Draymond Green    Klay Thompson    Stephen Curry
##               28               28               28               26
##     JaVale McGee       Nick Young     Kevin Durant   Andre Iguodala
##               24               28               28               28
## Shaun Livingston    Zaza Pachulia       David West
##               28               24               26
```

NBA passing

Closeness: weighted graph, weights as cost, take inverse

```
pass_close <- igraph::closeness(i_pass, weights = 1/E(i_pass)$weight,
                 normalized = T, mode="all")
sort(pass_close, decreasing = T)
```

```
##   Draymond Green     Kevin Durant    Stephen Curry    Klay Thompson
##        143.17652        133.70394        133.37731        123.29253
## Shaun Livingston   Andre Iguodala      Quinn Cook    Zaza Pachulia
##        114.50239        112.63231        108.26885        100.32001
##       David West      Jordan Bell     Kevon Looney       Nick Young
##         94.07287         92.34056         84.83873         82.55219
##    Patrick McCaw     JaVale McGee     Damian Jones
##         82.35205         59.57220         24.23892
```

NBA passing

Betweenness

```
pass_bet <- igraph::betweenness(i_pass, weights = 1/E(i_pass)$weight,
                 normalized = T)
sort(pass_bet, decreasing=T)
```

```
##   Draymond Green     Kevin Durant    Stephen Curry       Quinn Cook
##      0.434065934      0.153846154      0.148351648      0.142857143
##    Klay Thompson Shaun Livingston    Patrick McCaw      Jordan Bell
##      0.010989011      0.005494505      0.000000000      0.000000000
##     Damian Jones     Kevon Looney     JaVale McGee       Nick Young
##      0.000000000      0.000000000      0.000000000      0.000000000
##   Andre Iguodala    Zaza Pachulia       David West
##      0.000000000      0.000000000      0.000000000
```

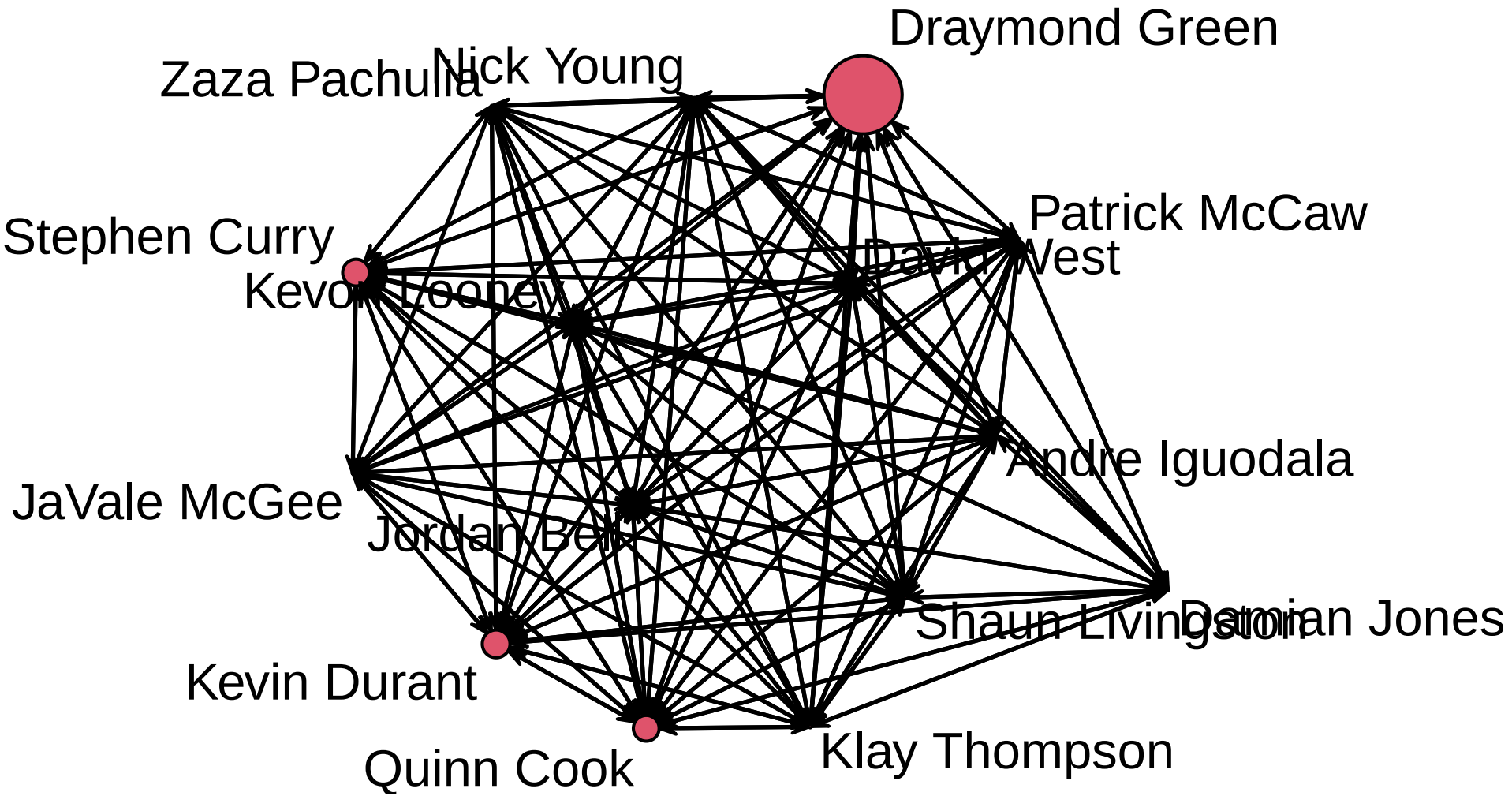
NBA passing

```
p_rank <- page_rank(i_pass)$vector
sort(p_rank, decreasing = T)
```

```
##   Draymond Green     Kevin Durant    Stephen Curry    Klay Thompson
##       0.13665118       0.12123153       0.10656721       0.10409694
##   Andre Iguodala Shaun Livingston       David West       Quinn Cook
##       0.07739258       0.07002230       0.05821080       0.05800035
##       Nick Young    Patrick McCaw    Zaza Pachulia      Jordan Bell
##       0.05171995       0.04812526       0.04717387       0.04173471
##     Kevon Looney     JaVale McGee     Damian Jones
##       0.03920867       0.02738581       0.01247885
```

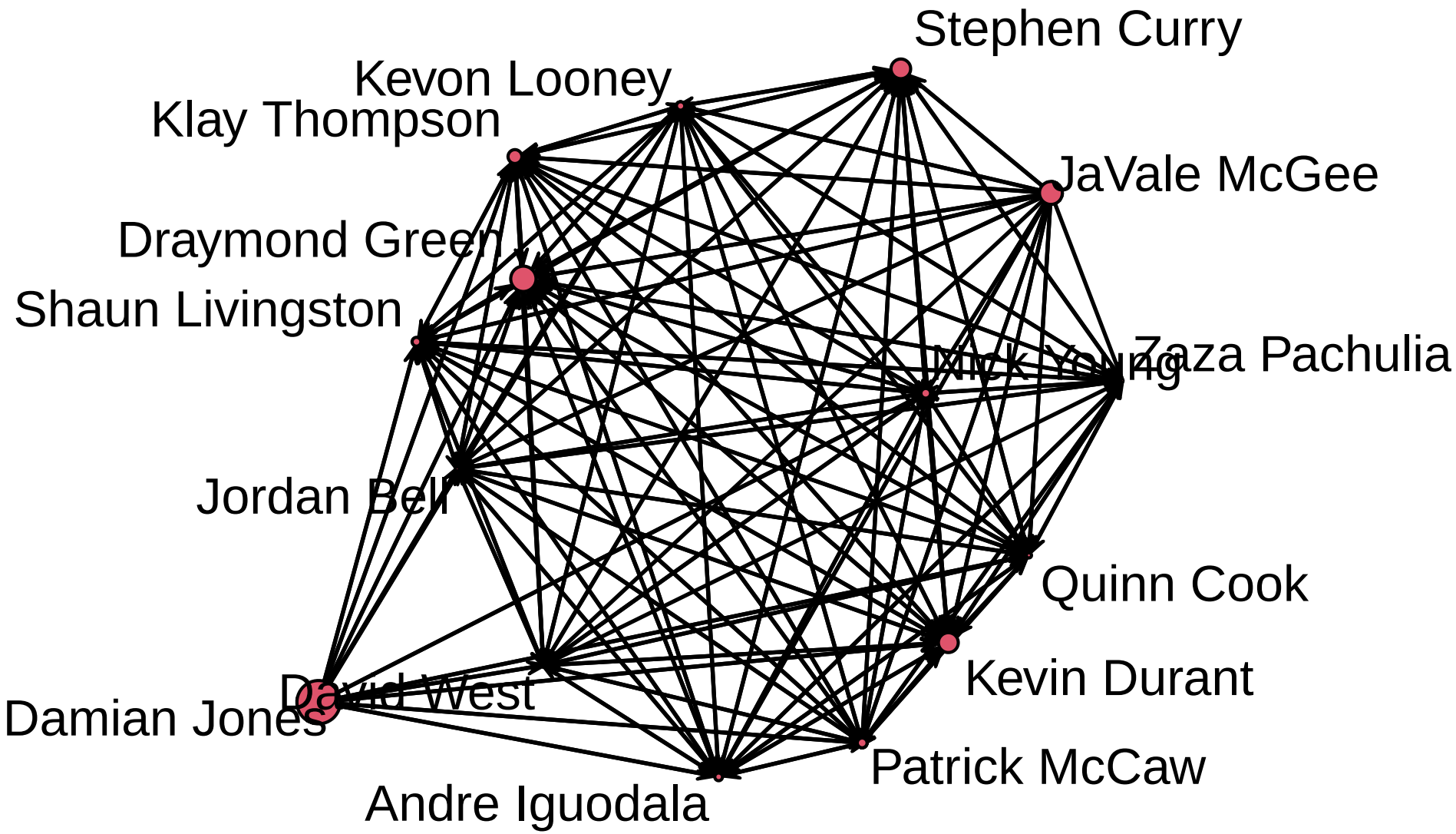
NBA passing

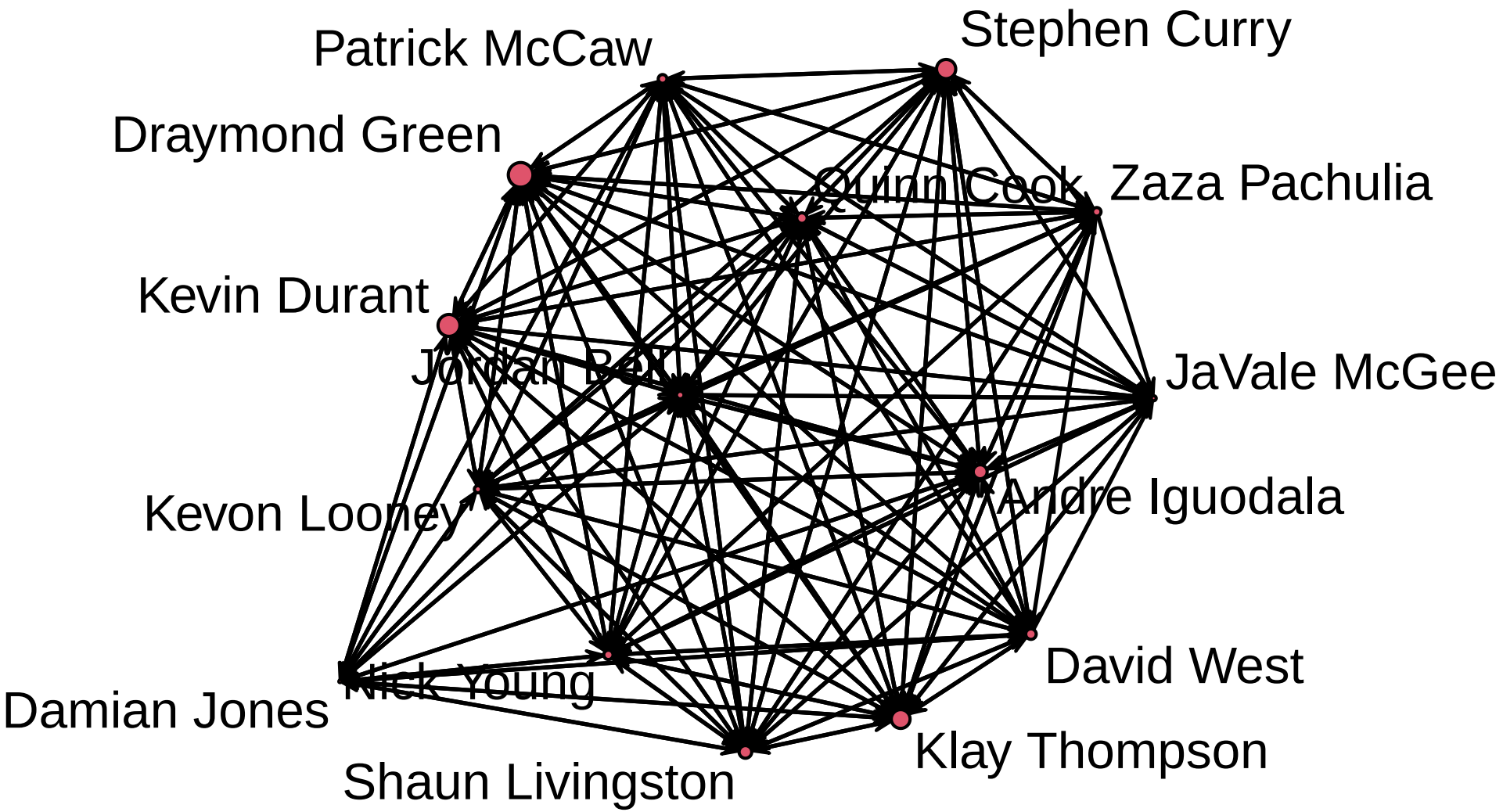
```
plot(n_pass, vertex.cex=10*pass_bet, displaylabels=T)
```



NBA passing

```
x <- (pass_close-mean(pass_close))/sd(pass_close)
plot(n_pass, vertex.cex=x, displaylabels=T)
```



NBA passing

```
plot(n_pass, vertex.cex=10*p_rank, displaylabels=T)
```